PGR PLATFORM - PRESENTER DOCUMENTATION

Demo Script

Second edition - the deep version.

Every screen gets two pages: what to do and say, then **every number on that screen** - what it is, exactly how it is computed, what it reads today, what value it adds, and the objections it attracts. Written so you can understand the platform yourself first, and deliver it second.

27	61	202	6	4
SCREENS	PAGES	METRICS DEFINED	DEMO LOGINS	LANDMINES NAMED

Metric definitions read out of the backend source. Values read off the captured screens and the live `pgr` database on 2026-08-27: 343 students, 344 applications, 994 milestones, 381 funding arrangements, 28 research awards, 12 model versions, 266 scored predictions. Where a number is an artefact of seeded data rather than a real result, this document says so and gives you the sentence to say.

How to use this document

THE STRUCTURE

Each screen has two facing pages. **Page one** is the walkthrough: what the screen is, the numbered steps, the line to say, and the screenshot. **Page two** is the anatomy: a table defining every component on the screen with its exact computation and its value today, then the value argument, the objections with answers, a caution, and where it goes next.

Read page two **before** the demo. Take page one **into** the demo.

HOW TO READ THE METRIC TABLES

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
The label as it appears on screen.	The actual rule, taken from the backend source - not a paraphrase. Where a rule has a threshold, the threshold and its tunable range are named.	What it reads right now, and a pointer if the number is a seed artefact.

WHAT CHANGED FROM THE FIRST EDITION

> The first edition told you what to click. It did not tell you what the numbers mean, so you could not answer the second question in the room. This edition defines every number, and it corrects **five factual errors** in the first edition that were verified against the running platform:

SCREEN	THE CORRECTION
Audit	v1 said reads as well as writes are captured. **They are not** - the middleware audits POST/PATCH/PUT/DELETE only. Reads are in the access log.
Progression	v1 told you to open a decided review and show the panel here. That is not on this screen - it lists milestone **definitions**. Decisions are on the student record.
Funding	v1 told you to show arrangements and the stipend schedule here. This screen lists funding **sources** only - two of them.
PGR Admin refusal	v1 said configuration tabs are not available to this role. The tabs render. The **server** refuses when you open one.
Executive	v1 said there is no route to student records. The sidebar still lists them. The **server** refuses the request.

THE THREE SENTENCES THE WHOLE DEMO RESTS ON

> **1. One person, one record, whole life.** The applicant becomes a student against the same person id, and Marcus Bell holds a student and an employee identity at the same time. Nobody ever runs a reconciliation.

> **2. Nothing is a black box.** Funding findings are named rules with dates. Supervisor matching attributes every point of the score. The model card computes its own limitations. Predictions carry per-student factors.

> **3. Governance is enforced, not documented.** A trainer cannot approve their own model. A reference value in use cannot be deleted. You cannot lock yourself out. Both AI safety switches are off by default.

PRESENTING POSTURE

> **Name the weak numbers before the room does.** This dataset was generated, and it shows in three places. In every case the rule is right and the sample is synthetic. Saying so first costs you ten seconds and buys you the room; being caught costs you the meeting. The landmine page overleaf gives you the exact wording.

> **Show refusals, not menus.** The strongest ninety seconds in this demo are two supervisor logins showing different caseloads, and an executive being refused a student record. Neither needs a slide.

> **Be straight about the boundary.** This is not a grants management system, not finance or payroll, not an HR system and not an undergraduate SIS. Saying that out loud is why the rest of the claims get believed.

Four numbers that will be challenged - and what to say

Each of these is on a screen you are going to show. In every case the logic is defensible and the data is not. Say the sentence in bold **before** anyone asks.

1 - CONVERSION RATE READS 99.7%

Where: Executive dashboard, and again as a Pattern Lab lock.
Why: the metric is converted applications over all applications - 343 of 344. The seed generated a student population and then wrote one application per student, so almost every application has a student behind it.
Say: “That number is an artefact of how the demo data was generated - it built students first and applications afterwards. The formula is right and the sample is synthetic. Watch what happens in Pattern Lab: the platform itself refuses to model applicant outcome, and the reason it prints is *conversion is ~99.7%, there is almost no negative class to learn from*. It caught its own data problem.”
Then move - do not linger. You have turned it into a governance point.

2 - FUNDED STUDENTS READS 2, OUT OF 343

Where: Executive dashboard tile.
Why: the tile counts distinct students with an arrangement whose `valid_to IS NULL`. In this platform `valid_to` is bitemporal - it marks a row that has been *superseded*. The seed gave every arrangement an explicit end date, so only 2 rows are open-ended even though 343 arrangements carry status `active`.
Say: “That tile is counting open-ended arrangements, and our generated data gave every arrangement an end date. 343 arrangements are active right now. It is a one-line predicate fix, and I would rather tell you that than have you find it.”
The fix is to test status and date coverage rather than `valid_to IS NULL`. If asked whether it will be fixed: yes, and it is small.

3 - ANALYTICS READS 264 AT RISK, 2 ON TRACK

Where: Analytics, and as the Funding column on every supervisor caseload row.
Why: the same predicate as above. One of the two deterministic risk rules is "no active funding", defined as no arrangement with `valid_to IS NULL` - so it matches 264 of 266 active students. The *other* rule is correct and meaningful: 38 students genuinely have an overdue milestone.
Say: “One of the two risk rules is firing on everybody because of the same open-ended-arrangement predicate. The rule that matters here is the milestone one - 38 students genuinely have an overdue milestone. And notice the platform still names the reason on every row rather than giving you a score, which is the actual point of this screen.”
Better still: for an executive audience, skip Analytics and show **Funding integrity** instead. It makes the same argument - deterministic, named, explainable - with numbers that are entirely sound (266 checked, 109 with findings, 97 errors, 25 warnings).

4 - THE PRODUCTION MODEL IS FLAGGED REVIEW, AUC 0.475

Where: Pattern Lab, Monitoring tab.
This one is not a problem - it is your best screen. The model scored 0.702 in training and 0.475 against what actually happened on 250 matured predictions, and the platform flagged itself and named both reasons.
Say: “This model is currently failing, and the platform is the thing telling you that. Every ML vendor shows you a model that works. Almost none will show you their product grading its own output as worse than a coin flip and refusing to hide it. The alternative product is one where this is equally true and nobody ever finds out.”
Do not apologise for it. Lead with it.

Before you start

1 - START THE PLATFORM

Run `start-all.bat` from the project root. It starts PostgreSQL, the API, the worker and the frontend, then opens the browser. Wait for the login page at `http://localhost:3000` .

> **Check before the room fills:** sign in as admin and confirm the dashboard shows **Platform connectivity - API reachable, Database ok**. If that card is green, everything downstream works. Then set browser zoom to **80%** - the relationship map and the wider tables need it.

2 - DEMO LOGINS

EMAIL	PASSWORD	ROLE	SEES
`admin@example.com`	`admin123`	Institution Admin + PGR Admin	Everything
`pgr.admin@example.com`	`pgr123`	PGR Administrator	Operations, no config
`rosa.grigsby@example.com`	`super123`	Supervisor	13 students
`elena.ford@example.com`	`super123`	Supervisor	2 students
`marcus.bell@example.ac.uk`	`student123`	Student	Own record only
`exec@example.com`	`exec123`	Executive	Dashboards only

3 - RECORDS THIS SCRIPT USES

WHAT	RECORD
Deep-dive student	`SCALE-99-0063` - Dara Hartley, start 2023-10-23, expected end 2027-04-23, predicted risk 5%
Concurrent identities	Marcus Bell - `PGR-2026-1586FA` - student and employee overlapping from 2026-09-01
Highest predicted risk	Sami Fontaine / Hugo Eriksen - 74%
Funding finding to read	Amara Abara `SCALE-99-0145` - 124-day funding gap
Production ML model	Funding Continuity Risk, v1, gradient boosting, currently flagged REVIEW
Delete refusal	Department **Computer Science** - 399 records in use

4 - TIMING

AUDIENCE	RUN	SCREENS
Executive (10 min)	Value only	Dashboard, Funding integrity, Pattern Lab Monitoring, Executive refusal. **Skip Analytics.**
Short (15 min)	Highlights	Dashboard, student record, Funding integrity, Pattern Lab Overview + Monitoring, two supervisor logins
Registry / PGR office (30 min)	Operations	Acts 1-7 and 9 - the whole lifecycle plus tasks, statutory and settings
Technical / governance (30 min)	Architecture	Student record, Audit, Integration, Statutory, all five Pattern Lab tabs, permission boundaries
Full (45 min)	Everything	Acts 1-10 in order

5 - FOUR THINGS TO AVOID

> **Do not open Supervision or My journey while signed in as admin.** That account is not linked to a person record, so both show an explanatory empty state and the moment is wasted. Use Rosa and Marcus - this script already does.

> **Do not run model training live.** The last runs took 29.5 s and 41.9 s. Run discovery if you must show something executing; otherwise use the pre-built results on the Models tab.

> **Do not present Completion or Award screens.** The dataset holds one completion record. If asked, say the lifecycle is built and the demo data starts at recruitment - and point at Pattern Lab locking its Completion Forecast target for exactly that reason.

> **Do not open Analytics to an executive audience.** See landmine 3. Use Funding integrity to make the same argument with sound numbers.

6 - ONE THING YOU MUST REHEARSE

> **Supervisor match has no captured results.** The screenshot in this document is the empty form. Run it once before the demo - type a proposal, press Suggest supervisors, and decide which supervisor you will read out and which factors you will add up.

The value argument in one page

WHAT THE PLATFORM REPLACES

TODAY, TYPICALLY	HERE
An applicant system and a student system, reconciled by a matching exercise that never fully succeeds	One person record. The student is created against the applicant's existing person id in the same transaction as the offer acceptance.
A student system and an HR system, with PGR students who also teach reconciled by hand	Concurrent dated identities on one person - 63 of the 343 researchers also hold an employee identity.
Funding lineage reconstructed from email and spreadsheets when a funder asks	Position linked to award at the moment of advertising; nine integrity rules over the whole chain in 0.09-0.17 s.
A statutory return implemented in code, changed by a release each year	Field mappings held as data, versioned by academic year, with a mandatory-field readiness gate before sign-off.
Board numbers assembled monthly into a pack that disagrees with the operational system	The Executive login reads the same six tiles from the same queries.
Analytics bought as a black box, then quietly not used because nobody can defend it	Named rules first; a governed ML layer beside them that grades and refuses itself.

WHERE THE HOURS ACTUALLY GO

> The three arguments that land hardest with a PGR office, in order: **(1)** funding problems found eighteen months early instead of by the student in the month they run out; **(2)** a statutory spec change that is a remap instead of a release; **(3)** a supervisor caseload that prioritises itself, so a supervisor with 13 students does not open 13 records to find the one who needs them.

MEASURED, NOT CLAIMED

CRITERION	TARGET	MEASURED
Standard read p95	< 300 ms	**6 ms**
Dashboard / report p95	< 2 s	**9-43 ms**
Cohort funding integrity, 266 students	usable at scale	**0.09-0.17 s** (was 1.2-1.9 s - a 14x fix after measuring)
Authentication cost per request	no DB round-trip	**0 queries** - claims embedded in the JWT
Concurrency	does not serialise	**20 concurrent readers** complete
Model training run	-	**29.5 s** and **41.9 s** for three algorithms with 5-fold CV

> Latency is measured **server-side** from the structured access log. On a single developer machine a client-side load generator competes with the API workers and Postgres and inflates timings by an order of magnitude.

HONEST ABOUT THE GAPS

> The non-functional criteria document deliberately marks things not done. The genuine gaps, in the order a buyer will care: **no TLS or encryption at rest configured** (deployment parked), **no endpoint rate limiting**, **no retention schedule**, **no subject-access export**, **no erasure/anonymisation**, **no tested backups**, **no CI vulnerability scanning**. None is an application rewrite; several need an institutional policy decision before they can be built correctly.

> **If asked what it takes to go live:** two virtual machines, PostgreSQL, an SMTP relay and a certificate. The infrastructure requirements and deployment guide are separate documents in `docs/`.

Executive dashboard `admin`

`/dashboard` · used by Pro-Vice-Chancellor (Research), Dean, Director of PGR

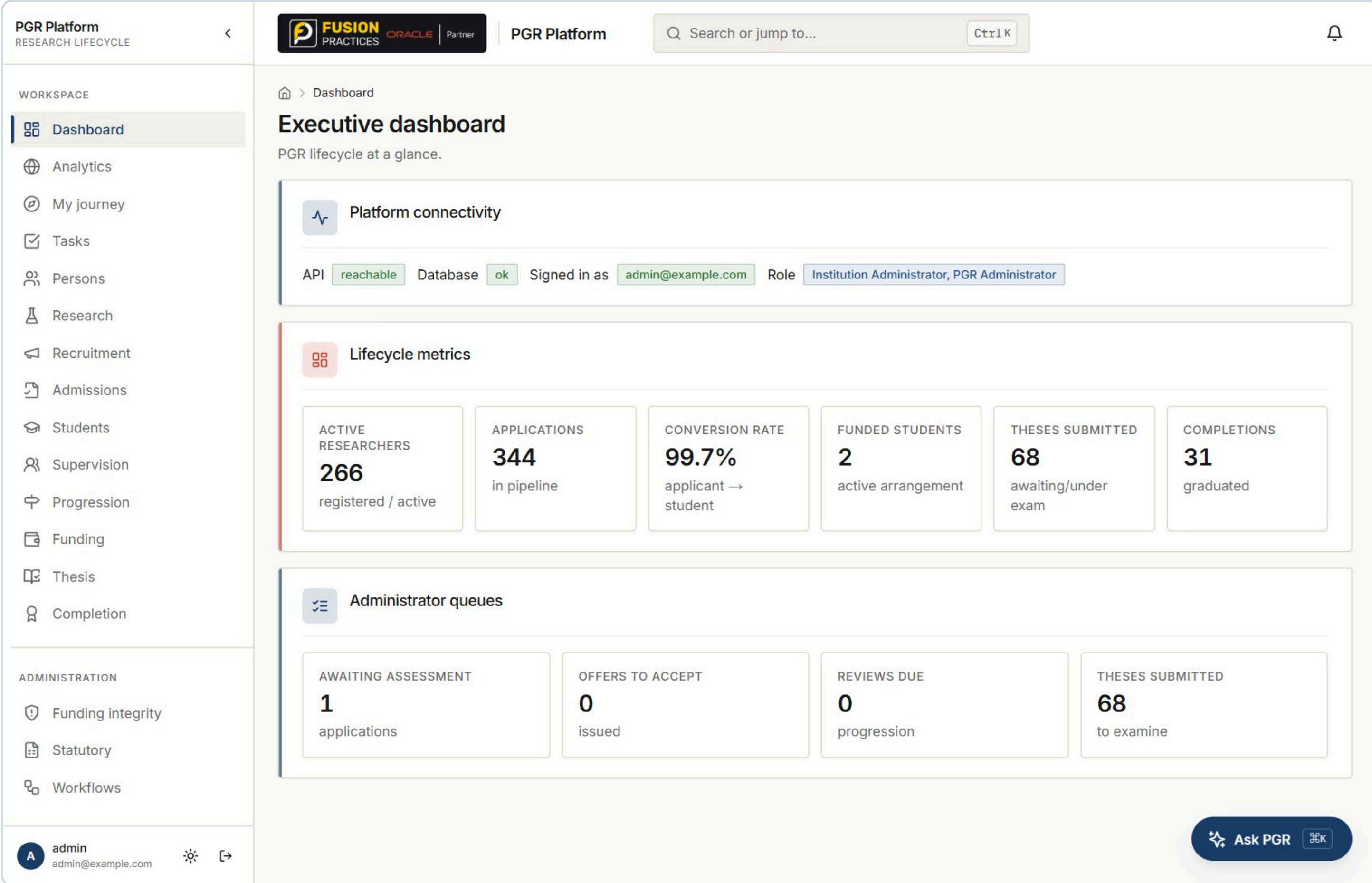
WHAT THIS SCREEN IS

The landing screen for every role. Three bands: a live connectivity check, six lifecycle metrics covering the whole applicant-to-alumni span, and four administrator queues that count work waiting on a human. Nothing is cached and nothing is a nightly batch - every tile is a SQL aggregate computed when the page loads.

WHAT TO DO

1. Sign in as **admin@example.com** / `admin123`.
2. You land here automatically - the route is chosen by permission, not by a menu click.
3. Read the connectivity card first: **API reachable**, **Database ok**, and the role chips. This is your proof the room is looking at a live system.
4. Walk the six lifecycle tiles left to right - they are the whole lifecycle in one row.
5. Drop to **Administrator queues**: this is today's work, not this year's statistics.

> *"Everything on this screen is computed live from 343 real student records when the page loads. There is no spreadsheet behind it and no overnight batch - if a registry clerk accepts an offer while we are talking, this number moves on refresh."*



Executive dashboard - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Platform connectivity	Two probes: `GET /health` for the API and a trivial SELECT for the database. The role chips are read from the JWT claims, not from a database lookup.	API **reachable** - Database **ok** - Institution Administrator, PGR Administrator
Active researchers	`COUNT(student)` where status is `registered` or `active`. Deliberately excludes suspended, withdrawn and completed.	**266** - from 199 active + 67 registered
Applications (in pipeline)	`COUNT(application)` across every stage - the total ever received, not the open ones.	**344** - 343 converted, 1 still at applicant stage
Conversion rate	applications at stage `converted` divided by all applications, x100, to 1 dp. "Converted" means the applicant became a registered student.	**99.7%** - 343 / 344. See the landmine page: this is a seed artefact, not a real yield
Funded students	`COUNT(DISTINCT student_id)` in `funding_arrangement` where `valid_to IS NULL`.	**2** - and 343 arrangements carry status `active`. See the landmine page
Theses submitted	theses at status `submitted` plus `under_examination`.	**68** - 29 submitted + 39 under examination
Completions	`COUNT(student)` at status `completed`. Note this is the student status, not the completion record - only 1 student has a `completion` row.	**31**
Awaiting assessment	applications at stage `applicant` plus `under_assessment`.	**1**
Offers to accept	offers at status `issued` - an offer sent but not yet answered.	**0** - 3 accepted, 1 still draft
Reviews due	milestones at status `submitted` plus `under_review` - a student has handed something in and a panel owes them a decision.	**0** - 810 decided, 182 due, 2 not started

THE VALUE IT ADDS

- **It replaces the Monday-morning spreadsheet.** In most PGR offices these six numbers are assembled by hand from a student system export, a finance report and a thesis tracker. That is typically half a day per month, per department, and the numbers disagree with each other because they were pulled on different days. Here they are one query against one database at the moment you look.
- **It separates statistics from work.** The top band is how big we are. The bottom band is what is waiting on a person. Most dashboards conflate the two and end up being read once a quarter and ignored.
- **It is the same screen for everyone.** The Executive sees the same tiles. There is no separate executive reporting build to maintain, and no risk of the board pack disagreeing with the operational system.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"How current is this?"	As current as the page load. Read p95 is 6 ms and the dashboard p95 is 9-43 ms, measured server-side. There is no refresh window to explain to anyone.
"Can a Head of School see only their school?"	Not yet - this is institution-wide. Row scoping already exists for supervisors and students, so a department filter is a scope rule, not a rebuild.
"Why is conversion 99.7%?"	Because the demo dataset was generated backwards from a student population, so almost every application has a student attached. Say it before they ask - the formula is right, the sample is synthetic.

Watch out. Two tiles will be questioned. Conversion rate 99.7% and Funded students 2. Both are explained on the landmine page - read that page before you present. Do not improvise an answer at the tile.

Where it goes next. Trend arrows against the previous period; click-through from each tile to the filtered list behind it; a department scope so a Head of School sees only their own population.

Recruitment - opportunities and pipeline `admin`

`/recruitment` · used by PGR Administrator, Research Office

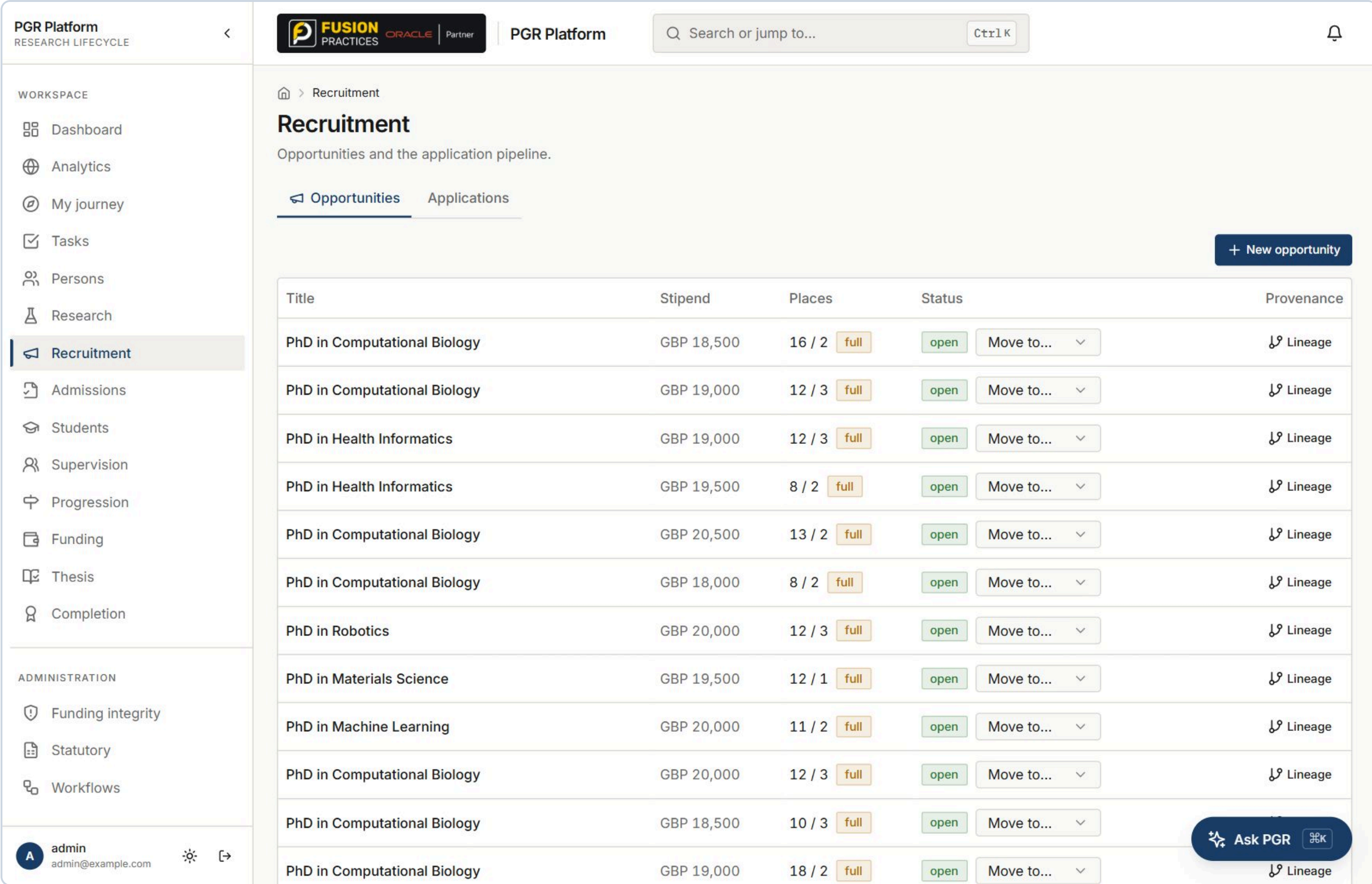
WHAT THIS SCREEN IS

Two tabs. **Opportunities** is the advertised position list - each row carries the stipend, the places offered against places filled, its open/closed state, a stage-move control, and a **Lineage** link back to the award that pays for it. **Applications** is the candidate pipeline. This is the screen that makes the funding trace possible three years later.

WHAT TO DO

1. Open **Recruitment** in the sidebar - you land on Opportunities.
2. Read one row out loud: title, stipend, places, status.
3. Click **Lineage** on any row to show the award behind the position.
4. Switch to the **Applications** tab and show the stage distribution.
5. Use **Move to...** on one application to show a stage change is one click, and note it writes a `candidate_stage_history` row rather than overwriting the stage.

> *"The position is linked to the award that funds it from the first day it is advertised. That single link is what lets you answer, three years later, which grant paid for this researcher - without anyone reconstructing it from email."*



Recruitment - opportunities and pipeline - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Opportunity title	`research_opportunity.title` . Created either directly or from a research demand raised against an award.	26 opportunities, mostly PhD in Computational Biology, Health Informatics, Robotics, Materials Science, Machine Learning
Stipend	The advertised annual stipend held on the opportunity - the number a candidate reads in the advert, not the amount eventually arranged for the student.	GBP 17,000 - 20,500 across the 26 positions
Places (filled / offered)	Left number = applications linked to this opportunity. Right number = advertised places. `full` shows when applications meet or exceed places.	Every row reads **full** - e.g. 16 / 2, 12 / 3, 8 / 2
Status	`open` / `closed` on the opportunity. Independent of whether it is full - an over-subscribed position stays open until somebody closes it.	All 26 **open**
Provenance / Lineage	Walks `opportunity - research_demand - research_award - funder` and renders it. This is the same lineage engine the Funding integrity screen uses.	28 research awards, GBP 54.84 m total value, 2 funders (UKRI EPSRC, University Scholarship Fund)
Application stage	`application.current_stage` , one of applicant, under_assessment, offer_made, converted, rejected, withdrawn. Every change also appends a `candidate_stage_history` row.	343 converted, 1 applicant
Entry route	`application.route` - opportunity-led (answered an advert) vs student-led (brought their own proposal). Carried through to Enterprise 360.	Recorded per application; surfaced in the Analytics population view

THE VALUE IT ADDS

- **Funding lineage is created at the cheapest possible moment.** Attaching a researcher to a grant at the advert costs nothing. Reconstructing it in year three, for an audit, costs a research finance analyst several days per award and is frequently impossible. This is the strongest single argument on the screen.
- **The advert is a record, not a PDF on a website.** Places offered, places filled and stipend live as data, so "how many positions did we advertise and fill this cycle" is a query rather than a survey of departments.
- **Stage changes are history, not a status field.** A stage move appends to `candidate_stage_history` . When a rejected candidate complains, you can show what happened and when.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"Can applicants apply through this?"	Not yet. There is no public applicant-facing advert page - applications arrive through the admin screens or an inbound integration message. That is a named gap, not a hidden one.
"We already advertise on FindAPhD / jobs.ac.uk."	Keep doing that. The opportunity here is the internal record that ties the advert to the award. The external board is a publishing channel.
"Every position says full - is anything actually open?"	That is the seed data: it generated more applications than places for every position. Say so.

Watch out. There is no bulk shortlisting action and no interview scheduling in this build. If asked, interviews and panellists exist as tables (`interview` , `interview_panellist`) but the scheduling UI is not built.

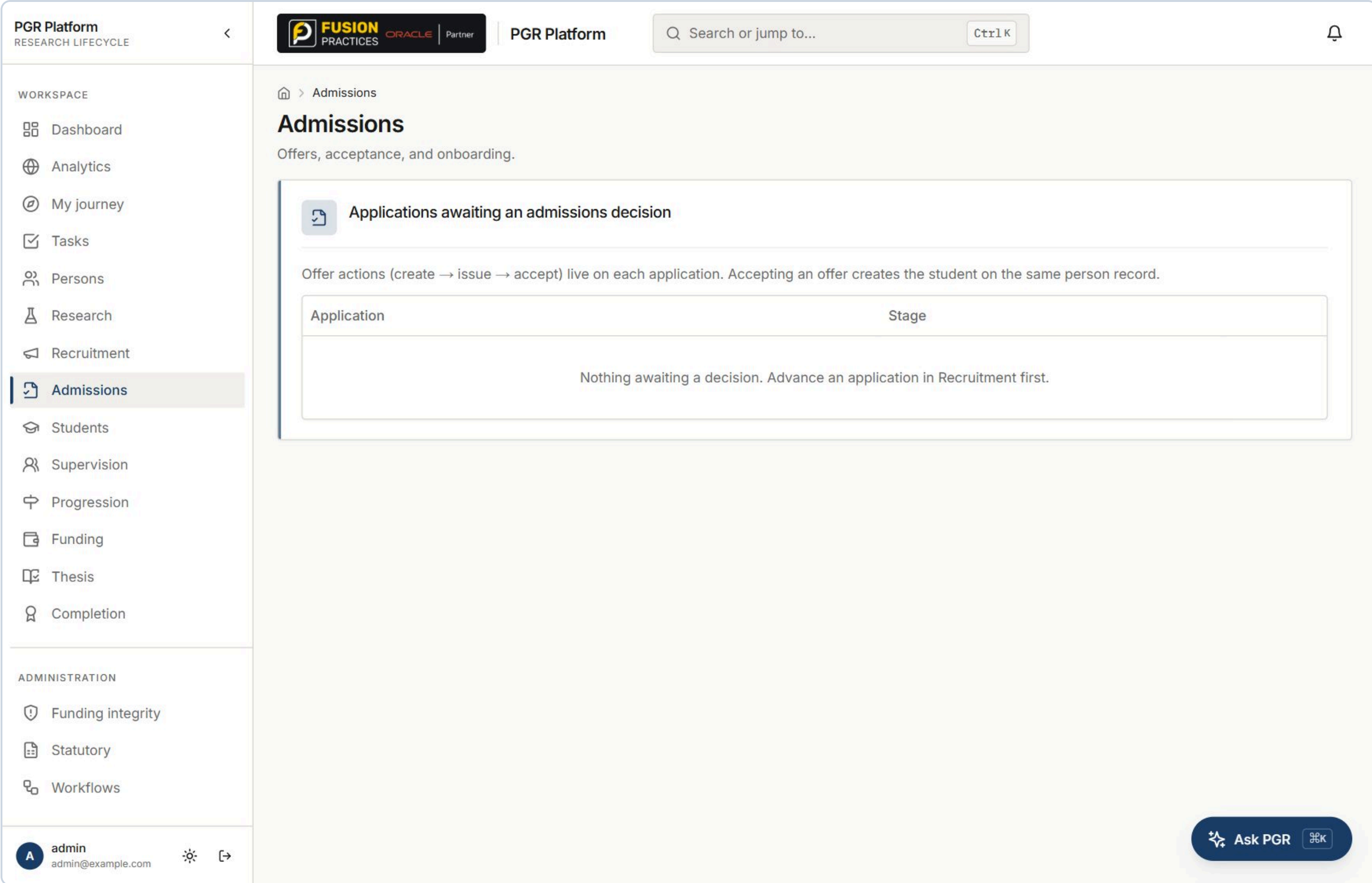
Where it goes next. A public applicant-facing advert and application form; bulk shortlisting; interview scheduling that syncs to Outlook or Google calendars.

Admissions, offers and conditions `admin`

`/admissions` · used by Admissions team, PGR Administrator

WHAT THIS SCREEN IS

Where a candidate becomes a commitment. An offer carries a status and a set of **conditions** held as first-class rows rather than free text, so "offer holder with outstanding conditions" is a state you can query. Accepting an offer is a single atomic transaction that creates the student, the identity relationship, the application stage change and the first tasks together.



WHAT TO DO

1. Open **Admissions**.
2. Open an offer and show its **conditions** as separate rows.
3. Explain what accept does - one transaction, four consequences.
4. Say the sentence about the person id explicitly. This is the platform's signature claim and this is where it is created.

> *"When this offer is accepted, the platform creates the student record against the same person record the applicant already had. The applicant and the student are never two people, so nobody ever has to reconcile them later."*

Admissions, offers and conditions - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Offer status	`offer.status` : draft, issued, accepted, declined, withdrawn. Only `issued` counts towards the dashboard's "Offers to accept" queue.	3 accepted, 1 draft, 0 issued
Offer conditions	Rows in `offer_condition` , each with its own description and met/unmet state. Not a text box.	Held per offer; drives the "outstanding conditions" question
Accept - what it does	One database transaction creates: the `student` row, a `person_relationship` of type student, the application stage move to `converted` , and the opening tasks. All four commit or none do.	Verified by `R7` in the non-functional criteria - all-or-nothing tested
Person continuity	The student is created with the applicant's existing `person_id` . No new person is minted, so identity threads stay unbroken through the whole lifecycle.	410 persons behind 343 students and 344 applications - the gap is staff and examiners, not duplicate applicants

THE VALUE IT ADDS

- **The reconciliation project that never has to happen.** In a split applicant system / student system estate, every institution runs a matching exercise to reunite the applicant with the student, and it never fully succeeds. Here the question does not arise, because the record was never split.
- **Conditions become manageable work.** "Who has an unmet condition two weeks before registration" is one filter, not an email thread with the admissions inbox.
- **Atomic acceptance removes a whole class of support tickets.** Half-created students - a student row with no identity, or an application stuck at offer_made - are the most common data-quality tickets in student systems. They cannot happen here.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"Can the applicant accept online?"	Not yet. Acceptance is recorded by an administrator. Applicant self-service and document upload against conditions is on the roadmap, not in the build.
"Does it generate the offer letter?"	No. Offer-letter generation from a template is a named gap. The offer and its conditions are recorded; the document is produced outside.
"What if the same person applies twice?"	There is a `person_merge_record` table and merge handling, so two person records can be reconciled with the merge preserved as history.

Watch out. The demo dataset holds only 4 offers, because the seed created students directly. If the room wants to see a rich admissions pipeline, this screen will disappoint - keep it short and spend the time on the person-continuity sentence, which is the real point.

Where it goes next. Offer-letter generation from a template; applicant self-service acceptance with document upload against each condition.

Student register `admin`

`/students` · used by PGR Administrator - this is the screen they live in

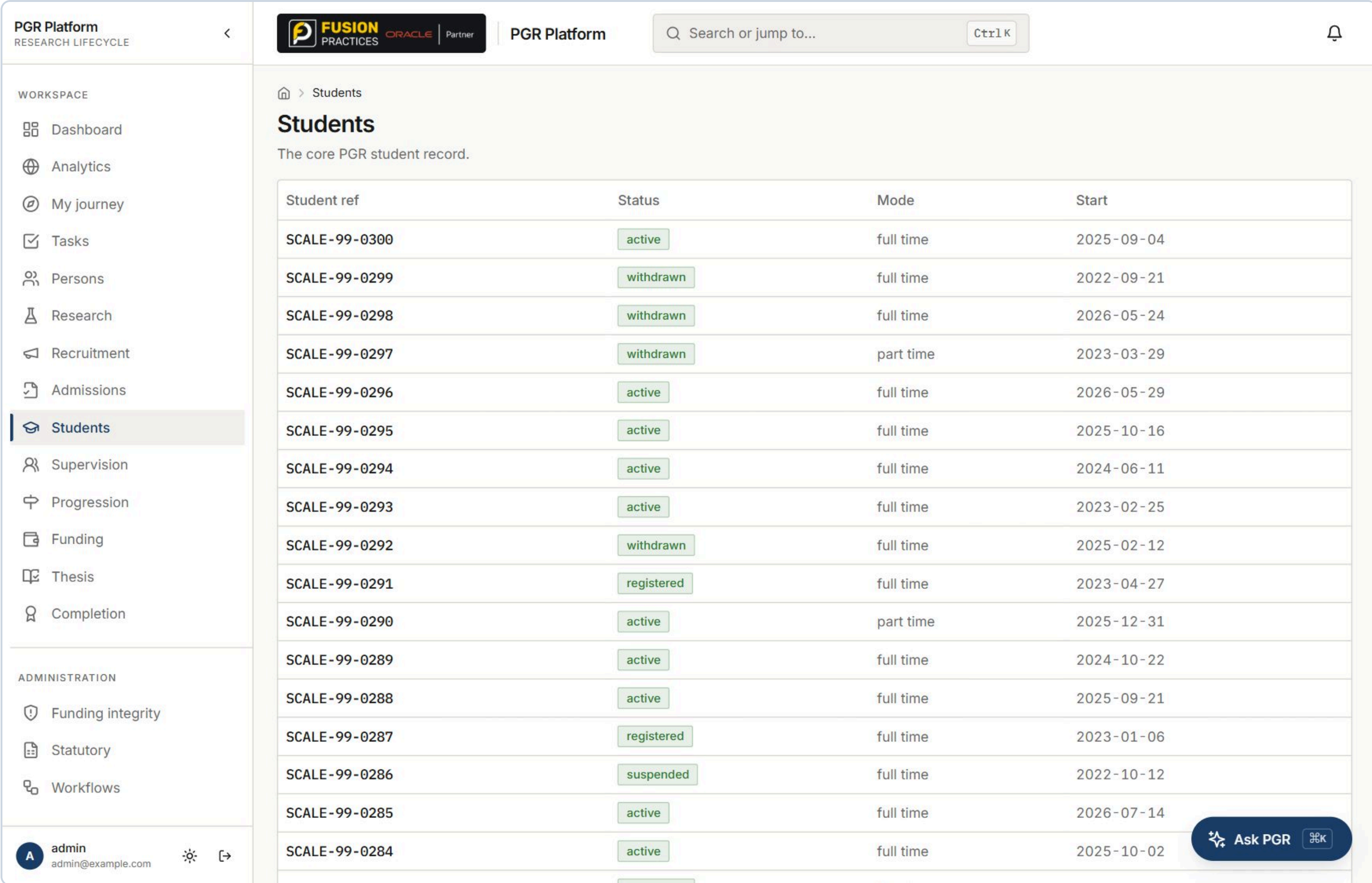
WHAT THIS SCREEN IS

The operational list. Every registered researcher, their reference, status and programme, with search and filtering. It is deliberately plain: this is the index, and the value is on the record behind it.

WHAT TO DO

1. Open **Students**.
2. Say the population size and the status mix out loud.
3. Search `SCALE-99-0063` and open **Dara Hartley**.
4. Make the point that every one of these arrived through the route you just walked - advert, application, offer, acceptance.

> *"343 students, and every one of them got here through the route we just walked. There is no back door that creates a student without a person and an application behind them."*



Student register - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Population	`COUNT(student)` , all statuses.	**343**
Status mix	`student.status` : active, registered, suspended, withdrawn, completed. Registered means enrolled and not yet started research proper; active means in study.	199 active - 67 registered - 33 withdrawn - 31 completed - 13 suspended
Student reference	`student.student_ref` , the institution-facing identifier. Two prefixes exist in the demo data because two seeds ran: `SCALE-99-*` (the 300-student cohort generator) and `UAT-42-*` (the hand-built demo set).	Deep-dive record is `SCALE-99-0063` - Dara Hartley
Suspended population	Students with an approved `suspension` lifecycle event currently in force. Their expected end date has been recalculated from the immutable registration baseline.	**13** suspended - 14 approved suspensions and 1 approved extension recorded

THE VALUE IT ADDS

- **One list, one truth.** The number here and the number on the dashboard are the same query. Institutions routinely run a registry list, a faculty list and a funding list that disagree by tens of students.
- **Withdrawn and completed stay in the population.** They are statuses, not deletions. Historic reporting, statutory returns and alumni tracking all need them, and none of them can work if a leaver is removed.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“Can I export this?”	Not from this screen yet. Statutory exports are generated from the Statutory profiles; a CSV export from an arbitrary filtered list is a named gap.
“Why two reference formats?”	Two seed scripts. In a real deployment the reference format is a single institutional rule. Say it plainly rather than hoping nobody notices.

Watch out. Saved views, bulk actions and CSV export do not exist. Do not promise them at this screen.

Where it goes next. Saved views (“my department, active, at risk”); bulk actions; CSV export from the filtered list.

The student record - the spine of the platform `admin`

`/students/{id}` · used by PGR Administrator, Supervisor, progression panel

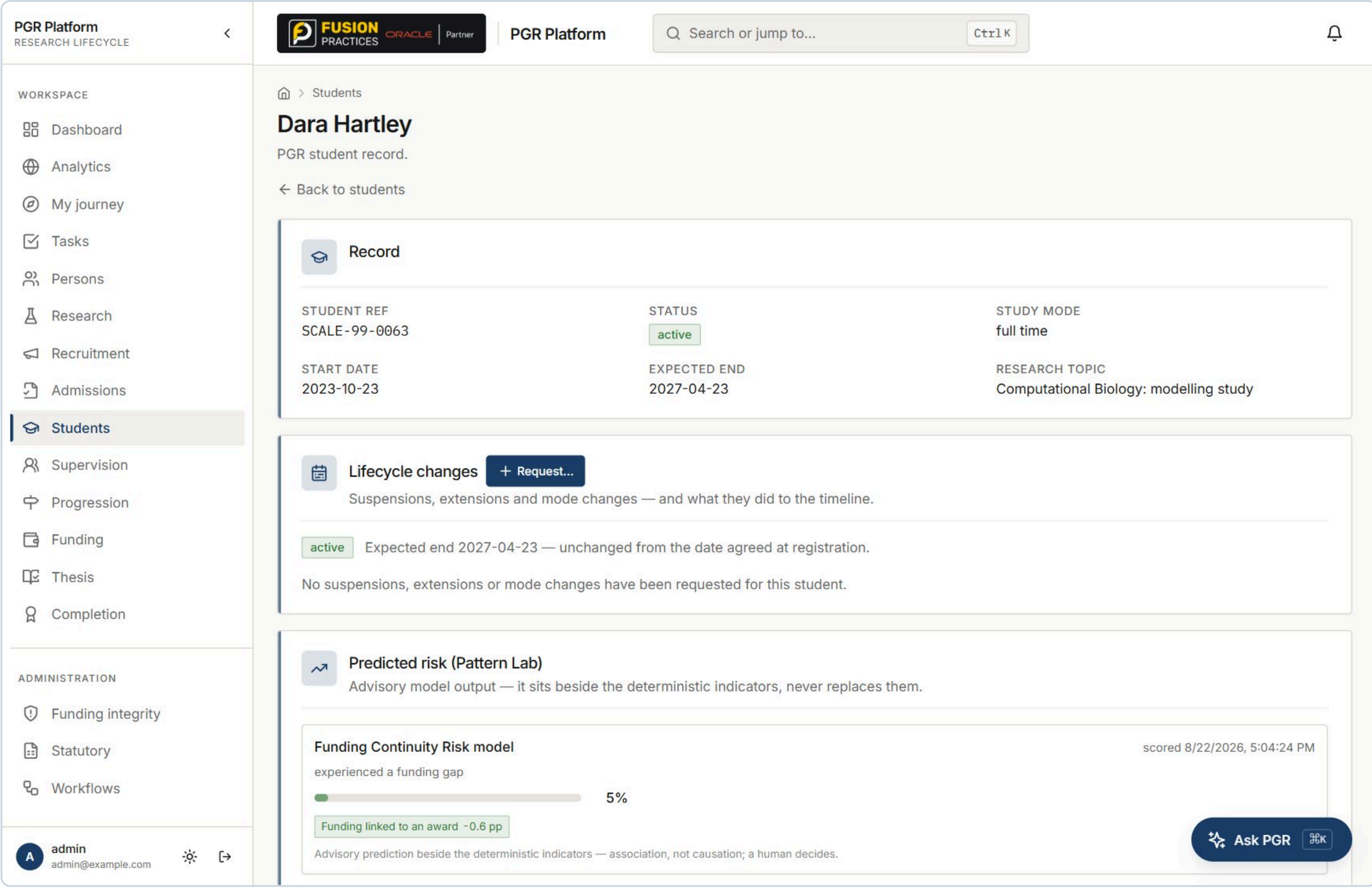
WHAT THIS SCREEN IS

The screen to slow down on. Record, lifecycle changes, the advisory prediction, supervision, milestones, funding arrangements and documents on one page. The Pattern Lab panel sits *beside* the deterministic facts, never instead of them, and carries its advisory wording on screen rather than in a footnote.

WHAT TO DO

1. From the register, open **Dara Hartley** (`SCALE-99-0063`).
2. **Record**: reference, status, study mode, start date, expected end, research topic.
3. **Lifecycle changes**: read the line “unchanged from the date agreed at registration” - then explain what would happen if a suspension were approved.
4. **Predicted risk**: read the probability, the contributing-factor chip, and the advisory sentence beneath it.
5. Scroll on through supervision, milestones and funding arrangements - this is where the funding history and the stipend schedule actually live.

> *“The expected end date is never typed in by a human. It is the registration date plus every approved suspension and extension, recomputed from an immutable baseline - so it can always be explained, and approving the same suspension twice cannot corrupt it.”*



The student record - the spine of the platform - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Expected end date	registration baseline + approved extensions + approved suspension days, recomputed from scratch each time. Idempotent and reversible - approve, reject and re-approve produce the same answer (criterion `R8`).	Dara Hartley: start 2023-10-23, expected end **2027-04-23**, unchanged - no lifecycle events
Lifecycle changes	`student_lifecycle_event` rows: suspension, extension, mode change. Each records the requester *and* the approver separately (criterion `A7`).	Across the cohort: 14 approved suspensions, 1 approved extension
Part-time rescaling	A mode change rescales the *remaining* registration period by the part-time factor. Past recalculations are never rewritten.	Factor **2.0**, tunable 1.0-4.0 in Institution policy
Predicted risk (Pattern Lab)	Probability from the production model version only. Contributing factors are computed per student, not copied from the model's global importances.	Dara Hartley **5%** - factor chip "Funding linked to an award -0.6 pp", scored 2026-08-22
Supervision	Current and historic `supervisor_relationship` rows - primary, co-supervisor, additional. A change closes one row and opens another.	Cohort-wide: 342 primary, 175 co-supervisor, 1 additional, across 59 distinct supervisors
Milestones	Instantiated from the programme's milestone definitions at registration, each with its own due date, status and - once decided - a `progression_review` holding the panel, the rationale, conditions and the appeal deadline.	994 milestones across the cohort: 810 decided, 182 due, 2 not started
Funding arrangements	Per-student `funding_arrangement` rows with valid_from/valid_to. A change closes the current row and opens a new one - it never overwrites.	381 arrangements across 343 students - 343 active, 36 ended, 2 changed
Stipend schedule	`stipend_payment` rows generated from the arrangement's frequency: 12 monthly, 4 quarterly, 3 termly, 1 annual, 1 one-off per year.	Sparse in the demo data - 1 paid (GBP 4,500), 3 scheduled (GBP 13,500)

THE VALUE IT ADDS

- **No tab-hopping between systems.** The single most common complaint about PGR administration is that answering one student question needs four systems. Everything a progression panel or a supervisor asks for is on this one page.
- **A date that can survive an appeal.** "Why is my end date this?" is answerable to the day, from an immutable baseline plus named approved events. A hand-edited date field cannot do that.
- **The prediction is placed, not promoted.** It appears below the facts, carries its advisory sentence, and names its factors for this student. That placement is the argument for why this ML is safe to put in front of a supervisor.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"Can the supervisor edit this?"	A supervisor sees their own students and can record meetings; they cannot change lifecycle dates. Row scoping is enforced at the server, so an out-of-scope student returns 404, not 403 - it does not even leak that the record exists.
"What if the model says 74% and the supervisor disagrees?"	Nothing happens. Predictions are advisory rows. Raising a task from a prediction is a setting and it is off by default.
"Can we print this for a panel?"	Not yet - a printable student summary is a named gap and one of the cheapest to build.

Watch out. For Dara Hartley the Lifecycle changes panel is empty ("no suspensions, extensions or mode changes"). If you want to show a recalculated end date, pick one of the 13 suspended students instead - find one before the demo, do not hunt live.

Where it goes next. A printable student summary PDF for progression panels; inline document preview so a reviewer never downloads; prediction history on the record.

Tasks, SLA and notifications `admin`

`/tasks` · used by PGR Administrator, Supervisor, anyone with a queue

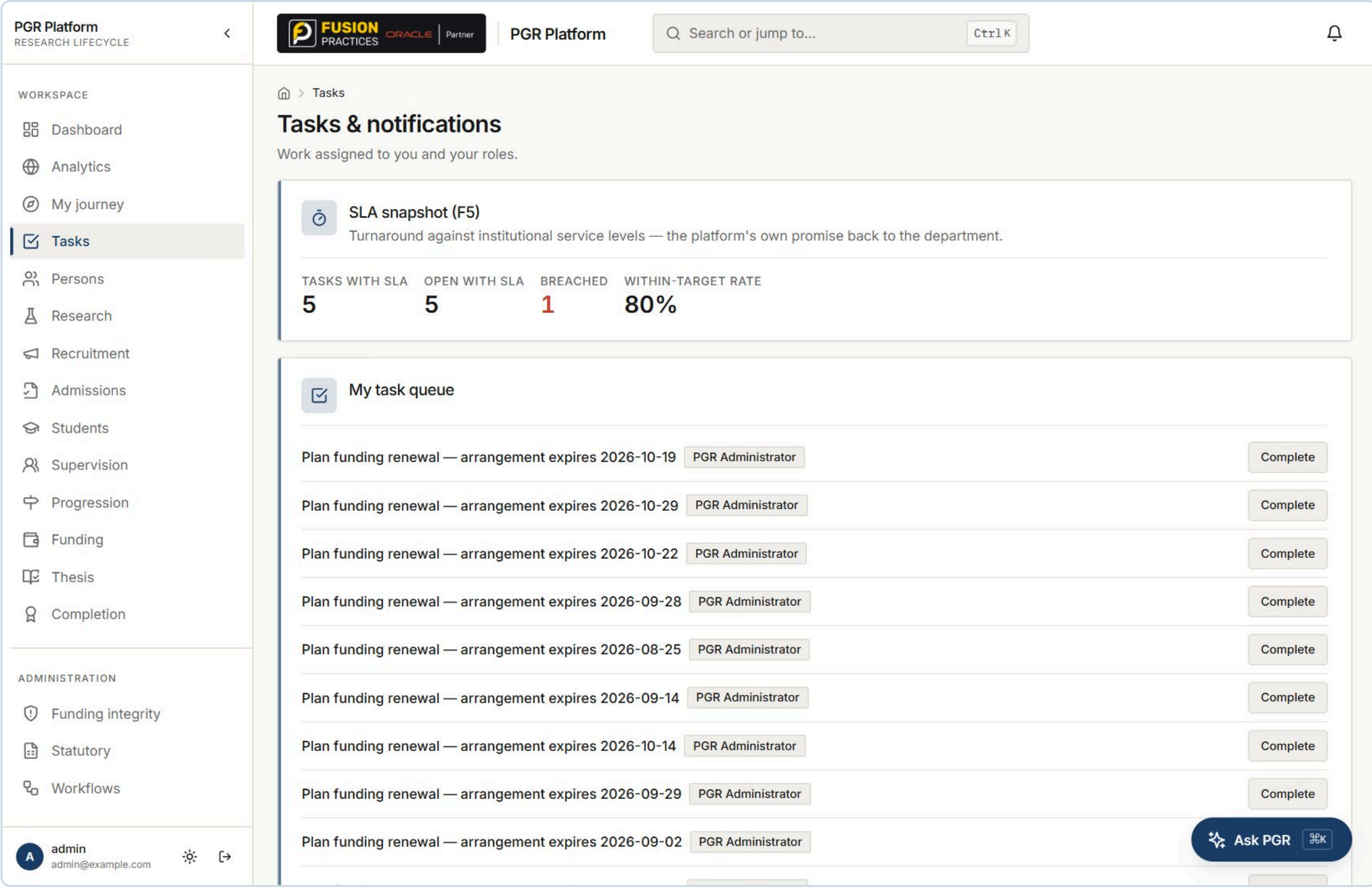
WHAT THIS SCREEN IS

Work reaches people instead of sitting in an inbox. The top band is the SLA snapshot - the platform's own promise back to the department, measured. Below it is the queue itself: every task carries the record it belongs to and the role it is addressed to, so acting on one is a click rather than a search.

WHAT TO DO

1. Open **Tasks**.
2. Read the **SLA snapshot** across: tasks with SLA, open with SLA, breached, within-target rate.
3. Point at the queue and note the tasks are addressed to a **role**, not a named person - so a leaver does not strand work.
4. Read one task title out loud: it names the arrangement and the date that caused it.
5. Say the sentence about nothing here being typed by a human.

> *"Nothing in this queue was created by hand. Each of these was raised by the platform when a rule fired - a funding arrangement is expiring, so somebody has to plan the renewal."*



Tasks, SLA and notifications - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Tasks with SLA	Tasks that carry a service-level target. An SLA is set per task type, not per task.	**5**
Open with SLA	Of those, the ones still open.	**5**
Breached	Open tasks past their SLA target, or closed later than target.	**1**
Within-target rate	(tasks with SLA - breached) / tasks with SLA, as a percentage. The platform grading itself.	**80%**
Task queue	`task` rows scoped to your roles. Each holds the entity it refers to, so the queue links straight back to the student or arrangement.	**21 open**, 2 done
Task provenance	Raised by the workflow engine or a scheduled job - e.g. “Plan funding renewal - arrangement expires 2026-10-19” comes from the funding-expiry job.	Every open task in the demo is a funding-renewal task
Notification preferences	Per user: email on/off, daily digest, quiet hours, and per-event mutes (milestone decided, task assigned, task escalated, funding expiring, thesis outcome, supervisor assigned).	Institution-wide email switch also exists in Institution policy and overrides personal settings

THE VALUE IT ADDS

- **The platform measures its own service, not just the students'.** A within-target rate is the number a PGR Director can take to a service review. Almost no student system reports on its own turnaround.
- **Role-addressed work survives staff turnover.** Tasks go to “PGR Administrator”. When someone leaves, nothing is stranded in a personal inbox - which is the single most common cause of a missed progression deadline.
- **Quiet hours and digests are a retention feature.** Supervisors switch notifications off when a system shouts. Per-event mutes and a digest are why they keep them on.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“Only 21 tasks for 343 students?”	Yes - because the demo dataset was seeded rather than operated. In use, tasks are raised by milestone due dates, funding expiry, offer conditions and thesis outcomes.
“Can I delegate or reassign?”	Not yet. Delegation, out-of-office reassignment and escalation to a manager are named gaps.
“Does it email?”	Yes, through a real SMTP relay, with bounces recorded in `email_bounce`. The institution-wide switch is in Institution policy.

Watch out. Only 5 tasks carry an SLA, so the 80% within-target rate is 4 out of 5. Do not present it as an institutional performance figure - present it as proof the measurement exists.

Where it goes next. Task delegation and out-of-office reassignment; a digest email summarising the queue; SLA breach escalation to a manager.

Supervisor caseload - row scoping made visible `supervisor`

`/supervision` · used by Supervisor, Director of Studies

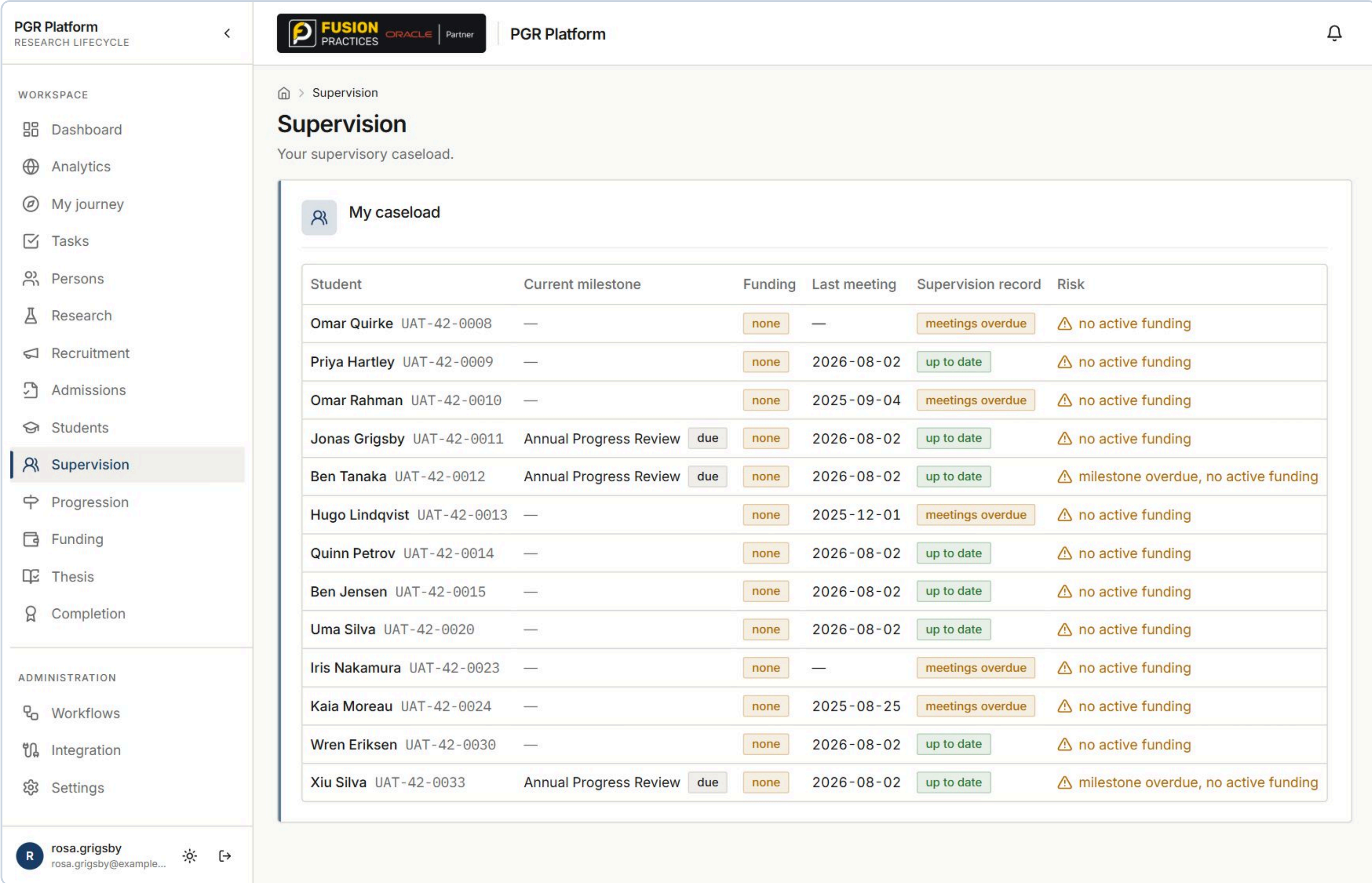
WHAT THIS SCREEN IS

The supervisor's own screen, and the clearest live proof of row scoping: the same URL shows a different world depending on who signed in. Each row puts current milestone, funding, last meeting, supervision-record state and risk on one line, so the student who needs attention is visible without opening anything.

WHAT TO DO

1. Sign out. Sign in as **rosa.grigsby@example.com** / `super123`.
2. Open **Supervision** - Rosa sees 13 students.
3. Point at the **Supervision record** column: "meetings overdue" vs "up to date", and at the **Last meeting** dates that produced it.
4. Sign in as **elena.ford@example.com** - the same screen, 2 students.
5. Say the sentence about the server deciding. If the room is technical, edit the URL to another supervisor's student and show the 404.

> *"Rosa sees 13 students. Elena sees 2. The server decides that, not the menu - and an out-of-scope student returns 404 rather than 403, so it does not even leak that the record exists."*



Supervisor caseload - row scoping made visible - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Caseload	`supervisor_relationship` rows for the signed-in person where `valid_to IS NULL`, resolved to students.	Rosa **13**, Elena **2**; 59 distinct supervisors across the cohort
Current milestone	The first milestone for that student whose status is not `decided`, with its definition name and due state.	e.g. “Annual Progress Review - due”
Funding	`active` if the student has any arrangement with `valid_to IS NULL`, otherwise `none`.	Reads **none** for almost every row - see the landmine page
Last meeting	Most recent `supervision_meeting` date for the student.	702 meetings recorded across the cohort
Supervision record	`meetings overdue` when the last meeting is older than the expected interval, otherwise `up to date`. The interval is an institution setting.	Threshold **90 days**, tunable 7-365
Risk	The deterministic rule, not the model: raised when the current milestone is overdue, or the student has no active funding. Both reasons are shown by name.	38 students cohort-wide have an overdue undecided milestone

THE VALUE IT ADDS

- **Security you can watch happen.** Two logins, thirty seconds, no slides. This is the most persuasive minute in the whole demo for an information governance audience - use it.
- **The caseload prioritises itself.** Milestone, funding, meeting recency and risk on one row means a supervisor with 13 students knows who to contact without opening 13 records.
- **Meeting cadence becomes evidence.** “We meet regularly” becomes a date and an interval. That is exactly what a progression appeal or a student complaint turns on.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“Could a supervisor just change the URL?”	They can try. Scoping is applied in the query, not in the UI - the row is never selected, so the response is 404. There is a test for exactly this.
“Can they record a meeting from here?”	Not from the row - meetings are recorded on the student record. Doing it inline is a named gap and an obvious one.
“Why does every row say no active funding?”	Data shape, not logic - the seed gave every arrangement an end date. Read the landmine page.

Watch out. Do not open Supervision as admin. The admin account is not linked to a person record, so the screen shows an explanatory empty state and the moment is wasted. Sign in as Rosa first. Also expect the Funding column to read `none` everywhere - have the landmine answer ready.

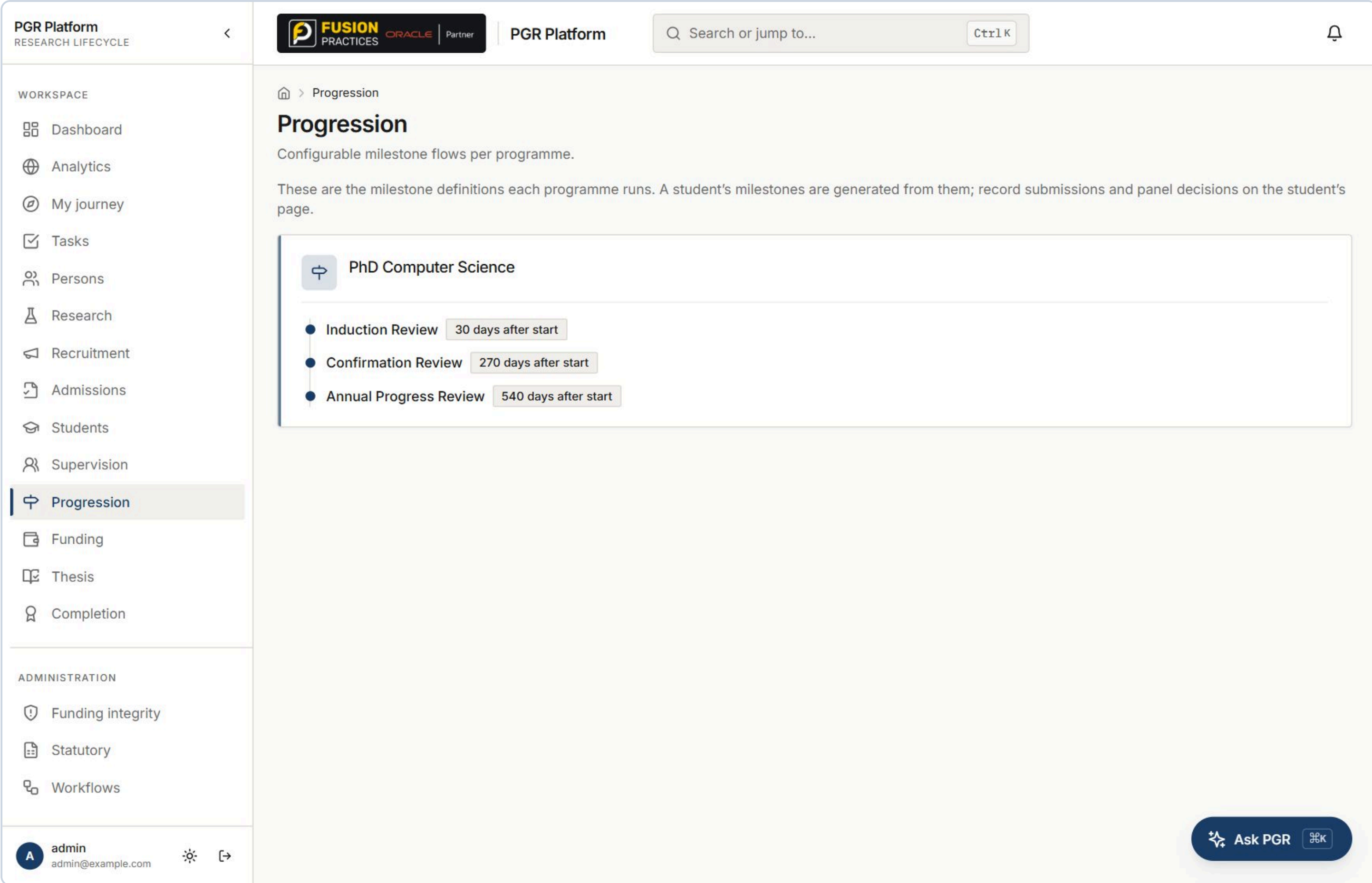
Where it goes next. Record a meeting inline from the row; a “needs my attention” sort; a supervisor digest email.

Progression - milestones as configuration admin

`/progression` · used by PGR Administrator, Graduate School

WHAT THIS SCREEN IS

This screen shows the milestone **definitions** a programme runs, with each one's offset from the student's start date. A student's milestones are generated from these at registration. Submissions, panel decisions, conditions and appeals are recorded on the student record, not here - be precise about that when you present it.



WHAT TO DO

1. Open **Progression**.
2. Read the three definitions and their offsets aloud.
3. Explain the consequence: change a programme's flow once and future students pick it up - you are not editing hundreds of student records.
4. Then say where the decisions live and, if the room wants to see one, go back to a student record and open a decided milestone there.

> *"These are definitions, not records. Change the flow once at programme level and every future student is generated from the new one - nobody edits three hundred milestone rows to make a policy change."*

Progression - milestones as configuration - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Milestone definition	`milestone_definition` per programme: a name and an offset in days from the student's start date.	PhD Computer Science: Induction Review **+30 d**, Confirmation Review **+270 d**, Annual Progress Review **+540 d**
Instantiated milestones	One `milestone` row per student per definition, created at registration with a due date computed from the offset.	**994** milestones: 810 decided, 182 due, 2 not started
Overdue	status is not `decided` and `due_date < today`. This is the rule that feeds the risk flag on Supervision and Analytics.	**38** distinct students currently overdue
Progression review	`progression_review`, holding supervisor assessment, panel decision, rationale, conditions, conditions-met date, re-review due date, appeal deadline and the outcome letter.	Recorded per decided milestone - shown on the student record
Panel membership	`review_panel_member` - who sat on the panel for that decision, kept permanently.	This is the row an appeal eighteen months later depends on
Appeal	`progression_appeal`, opened against a review within its appeal deadline.	Table exists and is wired; no appeals in the demo data

THE VALUE IT ADDS

- **Policy change without a data migration.** Adding a mid-year checkpoint to a programme is one definition row. In most student systems it is a configuration project plus a bulk update script.
- **An appeal-proof evidence trail.** Panel members, rationale, conditions and the appeal deadline are all stored. When a student appeals in eighteen months, everything the panel saw is still there, including who sat on it.
- **Overdue is one rule used everywhere.** The same definition drives the supervisor's row flag, the analytics risk list and the dashboard queue. There is no second definition of overdue to argue about.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“Where do I actually make a decision?”	On the student's milestone, not here. Say this rather than clicking around looking for it.
“Only one programme?”	The demo has one programme and one department. The model is per programme - the seed is thin, not the design.
“Do reviewers get reminded?”	Not automatically ahead of a due date. Reminder emails ahead of a milestone are a named gap.

Watch out. The v1 script was wrong here. It told you to “open a decided review and show the panel” on this screen. You cannot - this screen only lists definitions. Show the decision on the student record instead.

Where it goes next. Panel availability and scheduling; reminder emails ahead of a due date; outcome-letter generation from the recorded rationale.

Funding sources and arrangements `admin`

`/funding` · used by PGR Administrator, Research Finance

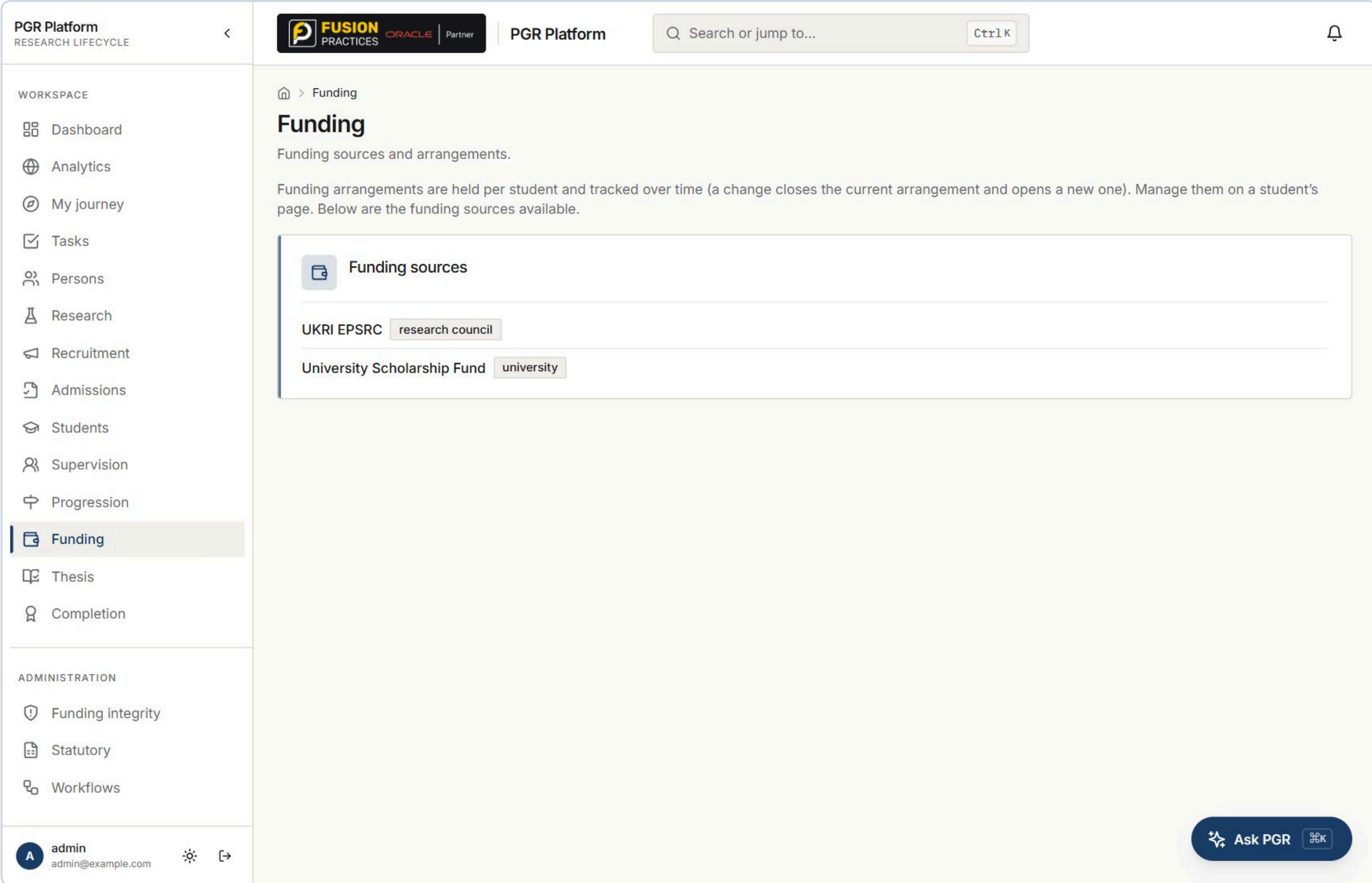
WHAT THIS SCREEN IS

This screen lists the **funding sources** the institution can draw on. The arrangements themselves - the relationship between a student, an award and a funder over time - are held per student and managed on the student record. The platform deliberately records the funding relationship and not payroll.

WHAT TO DO

1. Open **Funding** and show the sources.
2. Read the line at the top: arrangements are per student and change over time.
3. Go to a student record to show an actual arrangement and its stipend schedule.
4. Explain what a funding change does - closes one row, opens another, never overwrites.
5. Say the boundary sentence. Being clear about what you are not is worth more than another feature claim.

> *"We are not trying to be your finance system. We hold the relationship between a student, an award and a funder - and we keep every version of it, so the question is always answerable for a date in the past, not just today."*



Funding sources and arrangements - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Funding source	`funding_source` with a type: research council, university, external, self-funded. Administrator-managed reference data.	**2** - UKRI EPSRC (research council), University Scholarship Fund (university)
Funding arrangement	Per student: type, source, award link, stipend amount, currency, contribution %, valid_from, valid_to, status (planned / active / changed / ended).	**381** rows - 343 active, 36 ended, 2 changed
Bitemporal change	Changing funding sets `valid_to` on the current row and inserts a new one. The old row survives with its dates, so “who funded this student in March 2025” is answerable.	36 ended + 2 changed rows are exactly this history
Stipend schedule	Instalments generated from the arrangement's frequency: monthly 12/yr, quarterly 4, termly 3, annual 1, one-off 1. Each has a status: scheduled, approved, paid, held, cancelled.	1 paid GBP 4,500; 3 scheduled GBP 13,500
Committed spend	Payments in scheduled, approved or paid states count as committed against the award - the definition the integrity engine uses when it checks stipend against award value.	Award pool: 28 awards, GBP 54.84 m total value
Fee waivers	`fee_waiver` : full fee, partial fee or bench fee, held separately from the stipend.	Modelled; sparse in the demo data

THE VALUE IT ADDS

- **Point-in-time funding answers.** Auditors and funders ask about a date in the past. A system that overwrites the current funder cannot answer them; this one can, because the previous row is still there with its dates.
- **A defensible boundary.** Saying “we are not payroll” makes the rest of the claims credible. It also keeps the integration story simple: this system owns the relationship, finance owns the money.
- **The data the integrity engine needs.** Every check on the next screen - gaps, overlaps, funding outliving an award, stipend exceeding award value - is only possible because arrangements are dated rows rather than a current-state field.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“This screen is almost empty.”	Correct - and say so first. It lists sources; the arrangements are per student. Move to a student record rather than defending the screen.
“Does payment status come back from finance?”	Not yet. Outbound events go to a finance adapter; two-way return of payment status is the single most-requested integration and is a named gap.
“Can it forecast committed stipend spend?”	The data supports it - committed spend is already computed per award for the integrity check - but there is no forecast screen.

Watch out. This screen shows two rows. If you open it cold in front of a finance audience it looks thin. Open it, say the boundary sentence, and move immediately to a student record.

Where it goes next. Two-way finance integration so payment status flows back automatically; a forecast of committed stipend spend by year and by award.

Funding integrity engine admin

`/funding-integrity` · used by Research Finance, PGR Administrator, internal audit

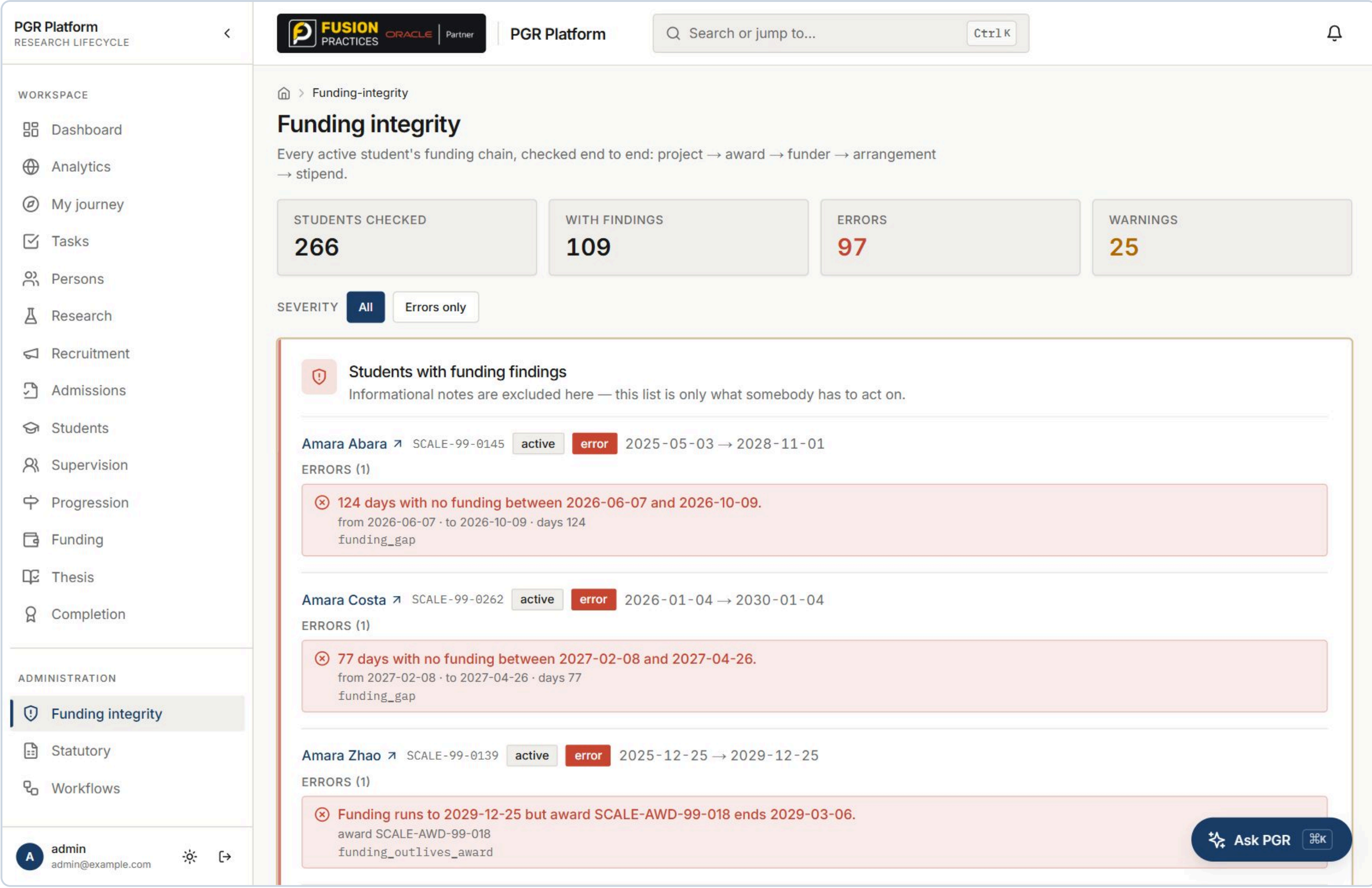
WHAT THIS SCREEN IS

The differentiator worth slowing down for. The platform walks every active student's funding chain - project, award, funder, arrangement, stipend - and names what is wrong. Nine named rules, each returning the dates or amounts that produced it. Deterministic: no model, no score, no guess.

WHAT TO DO

1. Open **Funding integrity** under Administration.
2. Read the four counters across the top.
3. Toggle **Errors only** to show severity is a real filter, not a colour.
4. Read one finding aloud in full - including the dates underneath it and the rule code.
5. Say the sentence about defending a finding to a supervisor.

> *"Every one of these has a name, a rule and the dates that produced it. Nothing here is a guess - which matters, because you are about to tell a supervisor their student has a 124-day funding gap."*



Funding integrity engine - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Students checked	Every active student, checked cohort-wide in one pass.	**266**
With findings	Students carrying at least one error or warning. Informational notes are excluded from this list - it is only what somebody has to act on.	**109** - 41% of the active population
Errors / Warnings	`error` = the record is wrong or the student will be harmed. `warning` = worth a look. `info` = expected, e.g. a self-funded student with no award link.	**97** errors, **25** warnings
`funding_gap` **(error)**	Consecutive arrangements with a gap longer than the tolerance. Reports the exact dates and day count.	e.g. Amara Abara - "124 days with no funding between 2026-06-07 and 2026-10-09"
`funding_ends_before_expected_end` **(error)**	Last arrangement ends before the student's expected end date, with the shortfall in days.	The most consequential rule - it is a student who will run out of money mid-study
`funding_outlives_award` **(error)**	Arrangement runs past the award's end date.	e.g. Amara Zhao - "Funding runs to 2029-12-25 but award SCALE-AWD-99-018 ends 2029-03-06"
`stipend_exceeds_award_value` **(error)**	Committed stipend (scheduled + approved + paid) exceeds the award's headline value.	The rule a research finance director cares about most
`funding_overlap` **(warning)**	Two arrangements overlap and their contribution percentages total more than 100%. Blended funding is legitimate; over 100% is not.	Reports the overlap days and the combined percentage
`unfunded_at_start` / `funding_precedes_award` **(warning)**	Funding starts later than the student did, or before the award it is charged to began.	Both report the day count
`no_project` / `project_not_linked_to_award` / `arrangement_award_unlinked`	Lineage completeness - the chain is broken, so spend cannot be attributed.	The last one fires when an arrangement carries a finance reference but no award link
Gap tolerance	`funding.min_gap_days` - gaps up to this are administrative noise (a weekend between arrangements), not a finding.	Default **7 days**, tunable 0-90 in Institution policy
Cohort performance	Reads each table once and runs the pure check in memory - cost is a handful of queries regardless of cohort size. The first implementation queried per student and was 14x slower.	**0.09-0.17 s for 266 students**, down from 1.2-1.9 s

THE VALUE IT ADDS

- **It finds money problems before the student does.** A funding gap discovered by the student, in the month it starts, is a complaint, a hardship case and often a suspension. Discovered eighteen months early it is an email.
- **Findings you can defend.** Because each carries its rule code and its dates, an administrator can forward one to a supervisor without having to justify a black box. This is the difference between a report people act on and a report people ignore.
- **It scales to the whole institution.** 0.09-0.17 s for 266 students, and the cost does not grow per student. Running it nightly across 3,000 researchers is not a capacity question.
- **41% with findings is the headline.** Not because the platform is noisy, but because this is what a funding estate actually looks like when somebody finally checks it end to end.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"97 errors - is the data that bad?"	It is generated data with deliberately messy funding, so the number is not an institutional statistic. What is real is that nine rules run over 266 students in under a fifth of a second and each finding names its dates.
"Can I dismiss a finding I have investigated?"	Not yet - dismissal with a recorded justification is a named gap, and it is the first thing an operational team will ask for.
"Does it raise a task automatically?"	Not yet. Auto-raising a task for error-severity findings is a named gap and a small one.
"Who decides the severities?"	They are fixed in the rule set today. A per-institution severity map is a reasonable ask.

Watch out. Read a finding in full, including the small grey line beneath it. The dates are the proof - reading only the red headline makes it sound like a score.

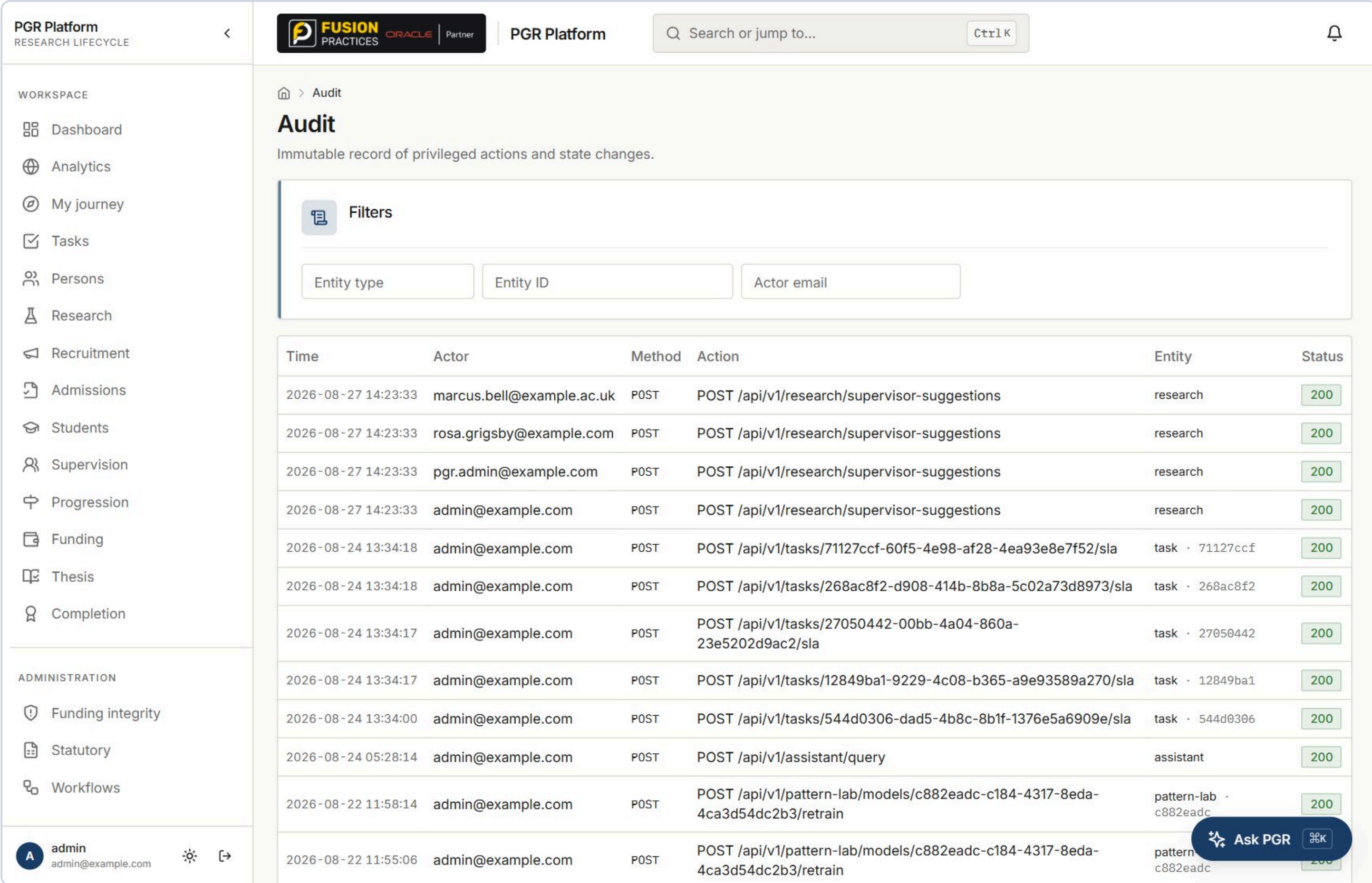
Where it goes next. Dismiss a finding with a recorded justification; auto-raise a task for error-severity findings; per-institution severity configuration.

Audit trail `admin`

`/audit` · used by Information governance, internal audit, DPO

WHAT THIS SCREEN IS

An append-only record of privileged actions and state changes, written by middleware at the edge of every request rather than by each endpoint. That is the only way it stays complete - no developer has to remember to add a line. Filterable by entity type, entity id and actor.



WHAT TO DO

1. Open **Audit**.
2. Filter by **Actor email** - type an address and show one person's activity.
3. Filter by **Entity type** - e.g. `task` or `pattern-lab`.
4. Point at the status column: the outcome is recorded, not just the attempt.
5. Say the sentence about middleware. The mechanism is the argument here, not the screen.

> *"No developer has to remember to write an audit line. It happens in middleware at the edge of every request, which is the only way an audit trail ever stays complete."*

Audit trail - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
What is audited	Every **state-changing** request: POST, PATCH, PUT, DELETE. Login and refresh are deliberately excluded as auth noise.	83 audit rows spanning 2026-08-21 to 2026-08-27
What is *not* audited	GET requests. Reads are captured in the structured access log with the same request id, not in this table.	Every row on screen is a POST - be accurate about this
Actor	Resolved from the JWT claims on the request, so there is no database round-trip and no way to act without being named.	e.g. `marcus.bell@example.ac.uk` , `rosa.grigsby@example.com`
Entity type and id	Parsed from the REST path - `/api/v1/tasks/{id}/sla` becomes entity `task` with that id.	task, research, assistant, pattern-lab all visible in the demo data
Outcome	The HTTP status code, so a refused attempt is recorded as clearly as a successful one.	200 on the visible rows
Request correlation	Every row carries the `requestId` that also appears in the access log, so an audit entry can be traced to its request (criterion `A8`).	Verified
Append-only	The application exposes no update or delete endpoint for `audit_log` . A database superuser could still edit it - true immutability needs DB-level grants.	Stated honestly as a known limitation (criterion `A5`)
Failure isolation	If the audit write fails, it is caught and logged - it never fails the user's request.	Criterion `A4` , tested

THE VALUE IT ADDS

- **Completeness by construction.** Per-endpoint audit code is always incomplete, because it depends on every developer remembering forever. Middleware cannot be forgotten. That architectural point lands with any auditor.
- **Requester and approver are both recorded.** For consequential decisions - a suspension, an extension, a model promotion - the platform records who asked and who approved, separately. That is what makes separation of duties real rather than declared.
- **It cuts the cost of an investigation.** "Who changed this student's funding and when" is a filter, not a forensic exercise across application logs.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"Are reads audited?"	**No - and do not say they are.** Writes are in this table; reads are in the structured access log. If the room needs read auditing on personal data, that is a real gap and worth capturing as a requirement.
"Can someone delete audit rows?"	Not through the application - there is no endpoint. A database superuser could. Making it genuinely immutable is a deployment control.
"How long is it kept?"	There is no retention policy yet. It needs an institutional decision before it can be built correctly, and it is marked as a gap.

Watch out. The v1 script was wrong here. It said "show that reads as well as writes are captured". They are not - the middleware only audits POST/PATCH/PUT/DELETE. Saying otherwise in front of an auditor is the one mistake in this demo you cannot recover from.

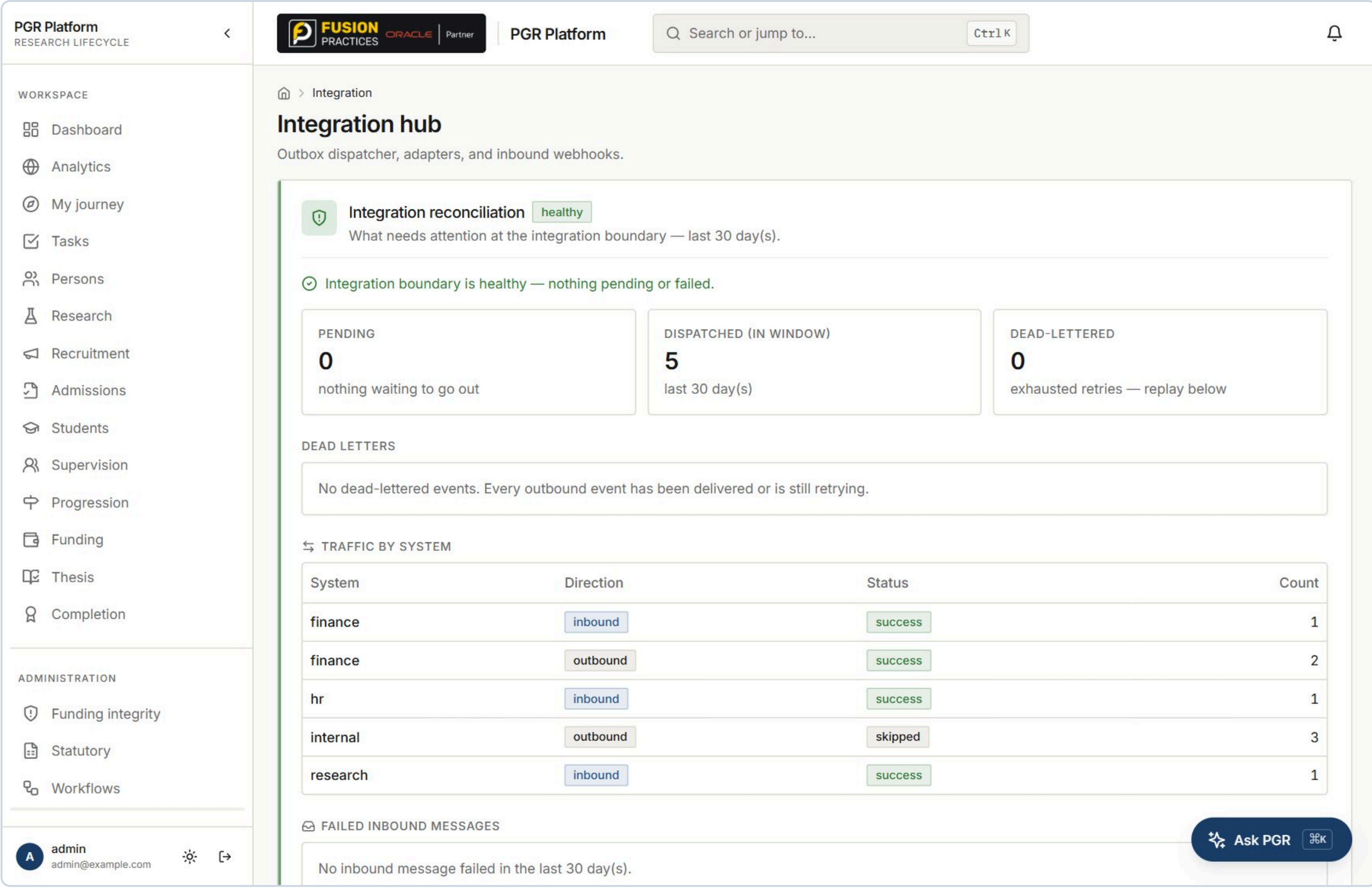
Where it goes next. Retention policy controls; export to the institution's SIEM; optional read-auditing for personal-data endpoints.

Integration hub and reconciliation admin

`/integration` · used by Integration team, PGR Administrator

WHAT THIS SCREEN IS

Answers the three questions an administrator actually asks at an integration boundary: is anything stuck going out, did anything fail coming in, and what is waiting on a person. Built on a transactional outbox - an outbound message is written in the same database transaction as the change that caused it.



WHAT TO DO

1. Open **Integration**.
2. Read the reconciliation verdict and its three counters.
3. Show **Traffic by system** - direction and outcome per partner system.
4. Show the dead-letter panel, empty, and explain what would be there and what replay does.
5. Say the outbox sentence. One line, and it is the whole resilience argument.

> *"An outbound message is written in the same database transaction as the change that caused it. So we can lose a network, and we can lose a partner system - but we cannot lose a message."*

Integration hub and reconciliation - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Reconciliation verdict	`healthy` when nothing is pending beyond its retry window and nothing is dead-lettered, over a rolling window.	**healthy** - last 30 days
Pending	`outbox_event` rows not yet dispatched.	**0**
Dispatched (in window)	Successfully delivered in the last 30 days.	**5**
Dead-lettered	Retries exhausted. These are visible and replayable rather than silently dropped.	**0**
Traffic by system	`integration_log` grouped by system, direction and status.	finance in 1 / out 2, hr in 1, research in 1, internal out 3 (skipped)
Retry and backoff	Failed deliveries retry with backoff (`attempts` , `next_attempt_at`), then dead-letter visibly with the last error recorded.	Tested end to end against a deliberately failing adapter
Inbound webhook security	Inbound messages must carry a valid HMAC signature over the raw body. A corrupted signature is rejected with 401 and is not recorded as a message.	Criterion `S7` , tested
Idempotency	A repeated partner message with the same `sourceId` is a no-op, not a duplicate.	Criterion `R5` , tested
Failed inbound	A malformed payload is recorded as `logged_with_error` - never silently dropped.	0 failed in the last 30 days

THE VALUE IT ADDS

- **No lost messages, and you can prove it.** The outbox pattern is the one design decision that removes “the finance system never got it” from the support queue permanently. Integration teams recognise it immediately.
- **Recovery without a developer.** A partner goes down, messages dead-letter, the partner comes back, an administrator replays from this screen. No ticket, no script, no database access.
- **Skipped is a visible state.** Three internal outbound events show as `skipped` rather than disappearing. Honest states are what make a reconciliation screen trustworthy.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“Only 5 events?”	Yes - the demo is seeded, not operated. The volume is not the point; the states and the replay path are.
“What systems does it integrate with?”	Adapters exist for finance, HR and research systems, plus generic inbound webhooks. There is a written integration contract in `docs/` .
“Will our partner know when something failed?”	Not yet - a partner-facing status page and retry alerting into an ops channel are named gaps.

Watch out. The worker and broker are endpoint-triggered stand-ins in this environment rather than a separate always-on process. If someone asks how the dispatcher runs in production, say it runs as a worker process - do not claim there is a message broker deployed here.

Where it goes next. A partner-facing status page; automatic retry alerting into the ops channel; per-partner SLA reporting.

Statutory returns as configuration `admin`

`/statutory` · used by Planning office, statutory returns lead

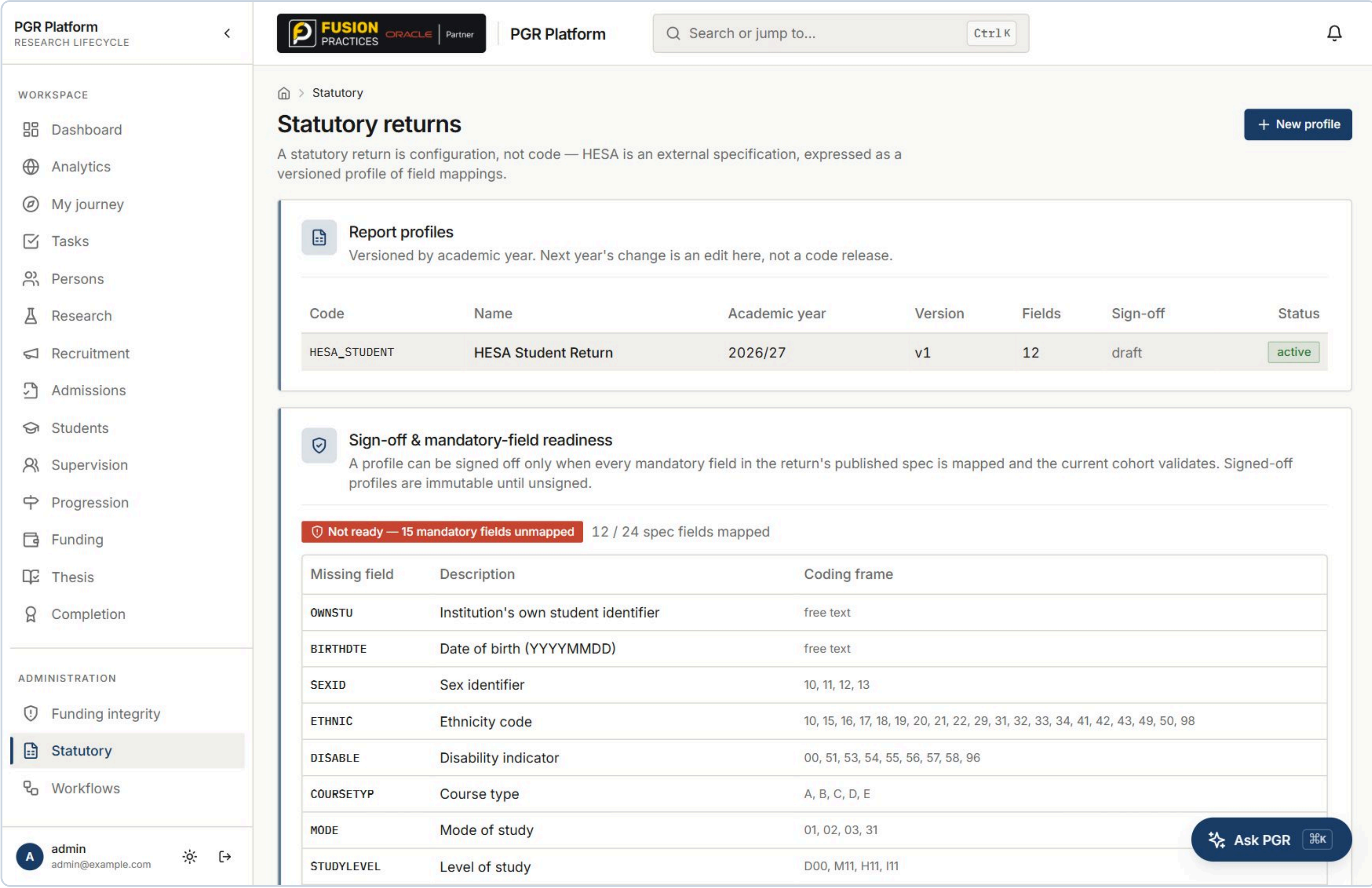
WHAT THIS SCREEN IS

A statutory return is treated as an external specification expressed as a versioned profile of field mappings - configuration, not code. The screen also grades its own readiness: it will not let a profile be signed off until every mandatory field in the published spec is mapped and the current cohort validates.

WHAT TO DO

1. Open **Statutory**.
2. Read the profile row across: code, name, academic year, version, field count, sign-off state, status.
3. Drop to **Sign-off & mandatory-field readiness** and read the red banner.
4. Scroll the missing-field table and point at the **coding frame** column - the platform holds the permitted values, not just the field name.
5. Say the sentence about next year's spec change.

> *"When the return specification changes next year, nobody here needs a developer. You clone last year's profile, remap the fields that moved, and the platform tells you what is still missing before you can sign it off."*



Statutory returns as configuration - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Report profile	`report_profile` : code, name, academic year, version, active flag, and sign-off fields. Versioned by academic year.	**HESA_STUDENT** - HESA Student Return - **2026/27** - v1 - active
Field mappings	`report_field_mapping` rows - one per return field, each pointing at a platform field or expression.	**12** mapped of **24** spec fields
Readiness verdict	A profile may be signed off only when every mandatory spec field is mapped *and* the current cohort validates.	**Not ready - 15 mandatory fields unmapped**
Missing fields with coding frames	The published spec's mandatory list, each with its description and permitted values.	OWNSTU, BIRTHDTE, SEXID (10/11/12/13), ETHNIC (20 codes), DISABLE (9 codes), COURSETYP (A-E), MODE (01/02/03/31), STUDYLEVEL (D00/M11/H11/I11) ...
Sign-off	`signed_off_by` , `signed_off_at` , `signed_off_notes` . A signed-off profile is immutable until it is unsigned.	**draft** - not yet signed off
Validate before generate	Validation runs against the current cohort before an export is produced, so errors surface before the file leaves the building.	Export jobs are tracked in `export_job`

THE VALUE IT ADDS

- **A spec change stops being a release.** HESA changes annually. In a coded implementation that is a change request, a development window, a test cycle and a release. Here it is an administrator remapping fields. This is the highest-value sentence on the compliance side of the demo.
- **The platform refuses to let you sign off something incomplete.** “Not ready - 15 mandatory fields unmapped” is the system protecting the person who signs. That is a governance feature, not a validation message.
- **Coding frames are held, not assumed.** Holding the permitted values means the export can be validated rather than merely produced - which is where most statutory return pain actually is.
- **Year-on-year comparability.** Because profiles are versioned by academic year and signed off, last year's return can be reproduced exactly as submitted.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“Is this the real HESA specification?”	**No.** It is a worked example carrying real HESA field names and coding frames to prove the mechanism. Shipping the current published spec as a starter profile is the single cheapest high-value item on the roadmap. Say this before they ask.
“Only 12 of 24 mapped?”	Yes - and the screen says so in red. That is the point of the readiness check. Do not apologise for it; present it as the feature.
“Can we compare two years?”	Not yet. A diff view between two academic-year profiles is a named gap.

Watch out. Do not let the room believe HESA is shipped and working. The mechanism is real and the field names are real; the mapping is a worked example. Being straight about this is why the rest of your claims get believed.

Where it goes next. Ship the current HESA specification as a starter profile; a diff view between two academic years; scheduled validation runs ahead of the return deadline.

Analytics - deterministic risk and completion `admin`

`/analytics` · used by Director of PGR, Graduate School, Head of Department

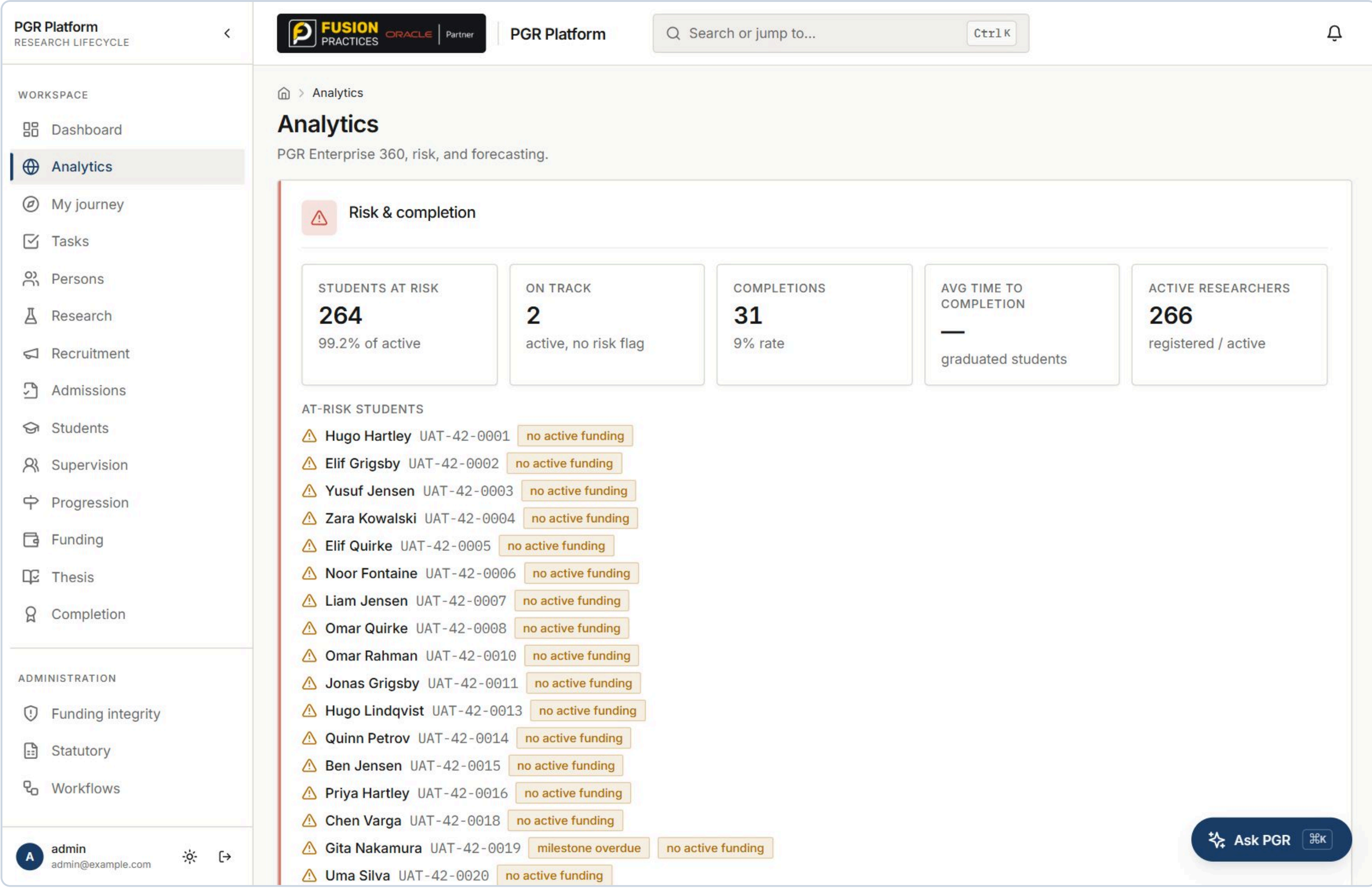
WHAT THIS SCREEN IS

The rule-based intelligence layer that exists *before* any machine learning. Risk indicators, completion analysis and the PGR Enterprise 360 population view - one population seen through five lenses: student, research, funding, workforce, statutory. Every number here is a rule you can read.

WHAT TO DO

1. Open **Analytics**.
2. Read the five tiles across.
3. Scroll to **At-risk students** and point at the **reason chips** - the reason is always named, never a score.
4. Scroll on to **PGR Enterprise 360** and explain the five lenses over one population.
5. Say the sentence about rules coming before models.

> *"Everything on this screen is a rule you can read out loud. The machine learning comes later in the demo, and it never replaces this - it sits next to it."*



Analytics - deterministic risk and completion - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Students at risk	Active students with at least one risk reason. Reasons are named on the row.	**264** - reported as 99.2% of active. See the landmine page
Risk rule: milestone overdue	The student's current undecided milestone is past its due date.	38 students genuinely match this
Risk rule: no active funding	No `funding_arrangement` with `valid_to IS NULL`. This is the rule producing the inflated count - it treats bitemporal supersession as coverage end.	Matches 264 of 266 active students
On track	active students - at-risk students. A projection, and the screen says so.	**2**
Completions / completion rate	students at status `completed`, over all students.	**31** - **9%** rate
Avg time to completion	Mean days from `student.start_date` to `completion.graduation_date`, over graduated students only.	**-** (blank) - only 1 completion record exists, and it lacks the dates to compute
Active researchers	Same definition as the dashboard - registered plus active.	**266** - the two screens agree, by construction
Enterprise 360 - five lenses	One population row per student carrying: student (status, mode, start, entry route), research (topic, group, area), funding (type, source, amount), workforce (is employee), statutory (nationality, programme, expected end).	343 rows; 63 of the population also hold an employee identity
Entry route	Opportunity-led vs student-led, taken from the application and keyed by person - because one person carries one identity thread.	Carried through from Recruitment

THE VALUE IT ADDS

- **Explainability before sophistication.** Every risk row names its reason. A supervisor can be told "your student's confirmation review is 40 days overdue" - which is actionable - rather than "the system rates them high risk", which is not.
- **Five questions, one population.** Registry, research office, finance, HR and planning normally each hold their own list of PGR students and each list is different. Enterprise 360 is the argument that they do not have to.
- **It sets up the ML honestly.** Showing the deterministic layer first, and saying the model sits beside it, is what makes Pattern Lab acceptable to a governance audience twenty minutes later.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"99.2% at risk - is that real?"	No, and say so immediately. One of the two risk rules is matching almost everyone because of how the seed data was written. Read the landmine page - you need this answer ready before you open this screen.
"Can we weight the risk rules?"	Not yet - configurable risk weighting per institution is a named gap. The meeting-interval and gap-tolerance thresholds are already tunable.
"Why is average time to completion blank?"	One completion record, and it lacks a graduation date paired with a start date. The platform shows a dash rather than inventing a number - which is itself the behaviour you want.

Watch out. This is the riskiest screen in the demo. It reads 264 at risk, 2 on track. Open it only after you have said the landmine sentence, or skip it for an executive audience and show Funding integrity instead - which is the same argument with correct numbers.

Where it goes next. Configurable risk weightings per institution; cohort comparison across intakes; fixing the active-funding predicate (see the landmine page).

Explainable supervisor matching `admin`

`/research (Supervisor match)` · used by Director of PGR, Head of Department, admissions panel

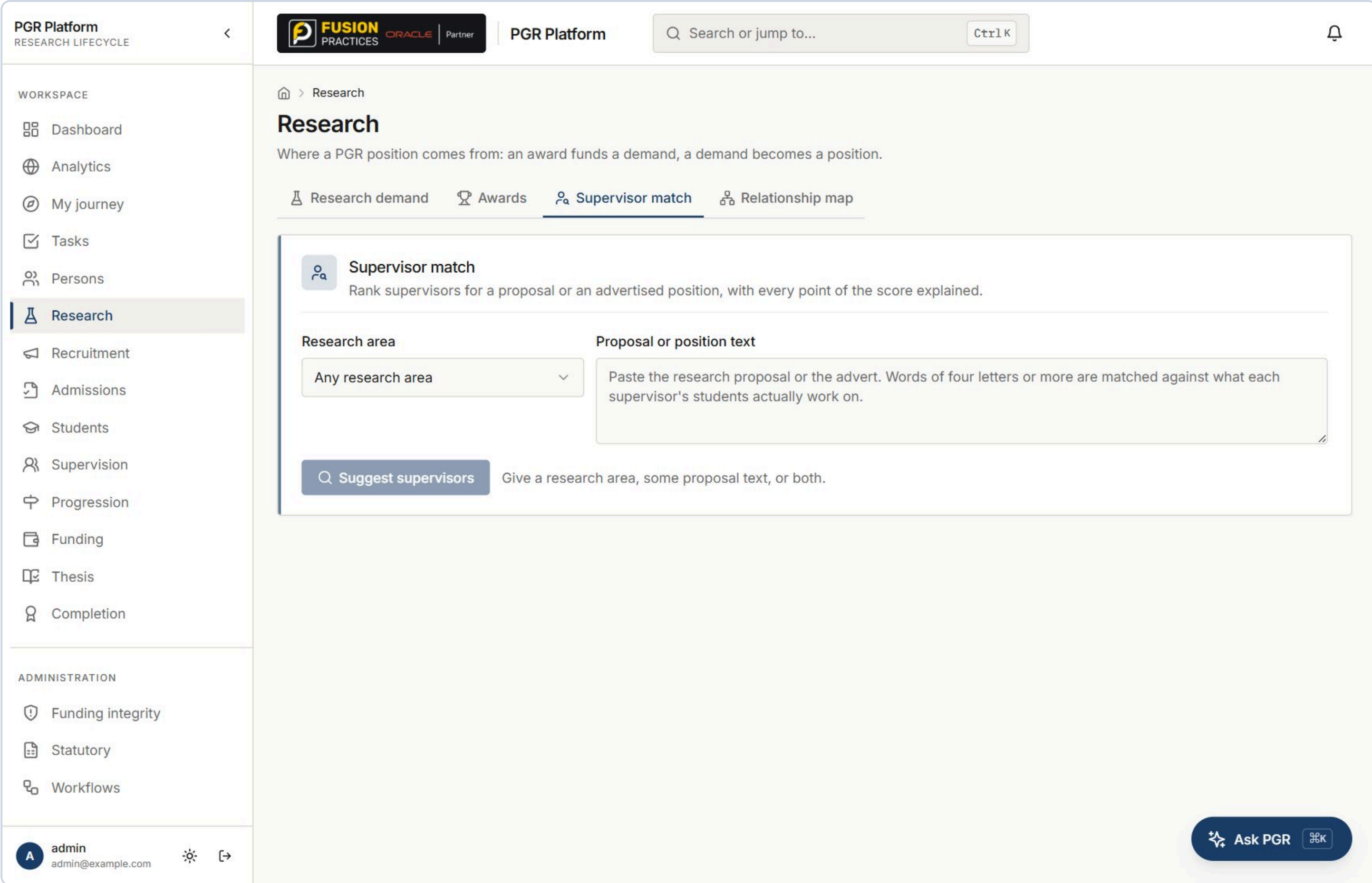
WHAT THIS SCREEN IS

Ranks supervisors for a proposal or an advertised position and shows exactly why each scored what it did. Deliberately not embeddings: a supervisor allocation is contested, so "why was I not suggested?" must have an answer a human can argue with. Four weights, small integers, summing to 100 so the score reads as a percentage.

WHAT TO DO

1. Open **Research** - **Supervisor match**.
2. Type a proposal, e.g. *machine learning for medical imaging*. You can also pick a research area, or both.
3. Click **Suggest supervisors**.
4. Read one supervisor's reasons out loud, factor by factor, and add them up to the score.
5. Scroll to a supervisor at capacity: they are still listed, scored down, never hidden.

> *"Every point in that score is attributed to a named factor - research area, topic overlap, capacity, track record. Add them up and you get the number. And a supervisor at capacity is scored down, never hidden, so the decision stays with a human."*



Explainable supervisor matching - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Research area (exact)	Supervisor already supervises in exactly this research area.	**45 points** - the dominant factor
Topic overlap	Words of four letters or more in the proposal, minus a stopword list, matched against the topics that supervisor's students actually work on. **8 points per shared term**, capped.	**up to 25 points** - the reason chip lists the shared terms
Capacity	`20 x min(1, free / (max_supervisees / 2))` where free = max - current. Full points once they have half the cap free.	**up to 20 points** - 0 at capacity, with the reason "at capacity (8/8)"
Track record	**5 points per completed supervision**, capped.	**up to 10 points**
Tie-break	Ties are common when only an area is given. Broken on who has more room, then on name - so the order is useful rather than alphabetical.	Deterministic and reproducible
Capacity source	`supervision.max_supervisees` from Institution policy - the same number that drives the assignment guard and the overdue flags.	Default **8**, range 1-50
Supervisor pool	People with a current supervisor relationship, with their area coverage, current load and completed count precomputed.	**59** supervisors; 342 primary + 175 co-supervisor current relationships

THE VALUE IT ADDS

- **It survives being challenged.** This is the whole design argument. An embedding-based recommender cannot answer "why not me". A four-factor additive score can, in one sentence, to a professor's face.
- **Capacity becomes visible at the moment of decision.** Supervisor overload is the most common cause of PGR attrition and the hardest thing to see. Here it is 20 points of a 100-point score, at the moment somebody is choosing.
- **It replaces the corridor conversation.** Allocation today happens by who a Director of PGR happens to think of. This surfaces the whole eligible pool - including people who are never thought of.
- **The weights are institutional policy, not vendor opinion.** They are four integers in one file. Making them tunable per institution is trivial - and worth offering when a room starts debating whether 45 is right.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"Why not use AI / embeddings?"	Because the output is contested. Embeddings would score better on paper and be indefensible in the room where the allocation is argued. That is a deliberate trade, not a limitation.
"Can we change the weights?"	Not from the UI yet - they are constants. Making them settings is a small change and an obvious one to offer.
"It does not know about publications."	Correct. Feeding publication records in as an additional, still explainable, factor is the named next step.

Watch out. The screenshot in this document is the empty form. There are no results captured. You **must** run this live - type a proposal and press the button. Practise it once beforehand and know which supervisor you will read out.

Where it goes next. Institution-tunable weights; publication records as an additional explainable factor; suggest-from-application so the panel gets a shortlist without retyping the proposal.

Research relationship map admin

`/research (Relationship map)` · used by Research office, PVC Research, grant audit

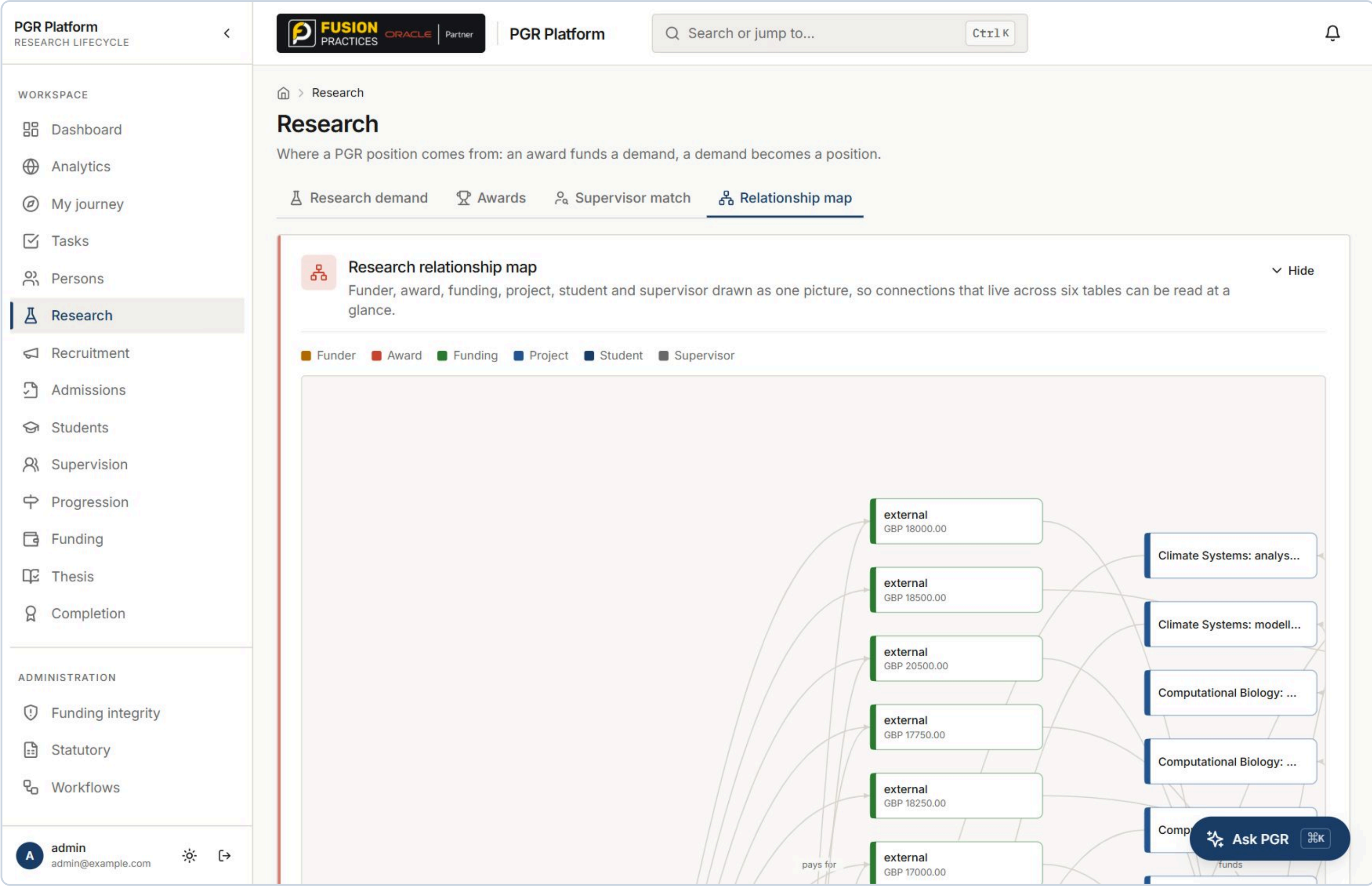
WHAT THIS SCREEN IS

Funder, award, funding, project, student and supervisor drawn as one picture. These connections live across six tables; this is the screen that makes them readable at a glance. Colour-coded by node type, with labelled edges - *pays for*, *funds* - and nodes that link through to the record.

WHAT TO DO

1. Open **Research** - **Relationship map**.
2. Read the legend: funder, award, funding, project, student, supervisor.
3. Trace one path left to right, out loud, following the edge labels.
4. Click a node to jump to that record.
5. Say the sentence about the grant question.

> *"This is the question 'who and what is connected to this grant' answered in one screen, instead of a research finance analyst joining six tables by hand."*



Research relationship map - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Node types	Six: funder, award, funding, project, student, supervisor - colour coded in the legend.	Rendered as a left-to-right layered graph
Funder layer	`funding_source` - the organisation the money comes from.	2 funders
Award layer	`research_award` - award reference, title, value, currency, start and end dates, source system.	28 awards, GBP 54.84 m
Funding layer	`funding_arrangement` nodes labelled with type and amount, e.g. “external GBP 18,500”.	381 arrangements
Project layer	`research_project` - the research topic, e.g. “Computational Biology: modelling study”.	One per student where recorded
Edges	Labelled relationships - *pays for*, *funds* - so the direction of the money is readable without a key.	Drawn from the same lineage walk as Funding integrity
Scoping	Can be centred on a student or on an award.	Both entry points supported by the API

THE VALUE IT ADDS

- **The grant audit question, answered visually.** “Show me everyone this award paid for” is a routine funder request and normally a multi-day reconstruction. Here it is a screen.
- **It makes the data model self-evident.** For a technical audience this one picture explains the whole domain model faster than the architecture document.
- **Impact reporting starts here.** Funder to award to researcher to output is the chain every impact case study needs. The first three links are already drawn.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“Can I filter it?”	Not yet - filtering by funder or department, and expanding outwards from a clicked node, are named gaps. Today it is centred on a student or an award.
“Will this be readable at 3,000 students?”	Not as a whole-institution picture, and do not claim it will be. It is a per-award and per-student view. Filtering is what makes it scale.
“Does it include publications or outputs?”	No. Research outputs are not in scope - that is the research information system's job.

Watch out. The graph is dense and the labels truncate. Zoom the browser to 80% before you open it, and decide in advance which single path you will trace. Do not pan around looking for a story.

Where it goes next. Filter by funder or department; expand the graph outwards from any node on click; export the traced path as evidence for a funder report.

Pattern Lab - the governed ML layer `admin`

`/pattern-lab (Overview)` · used by Director of PGR, data governance, institutional research

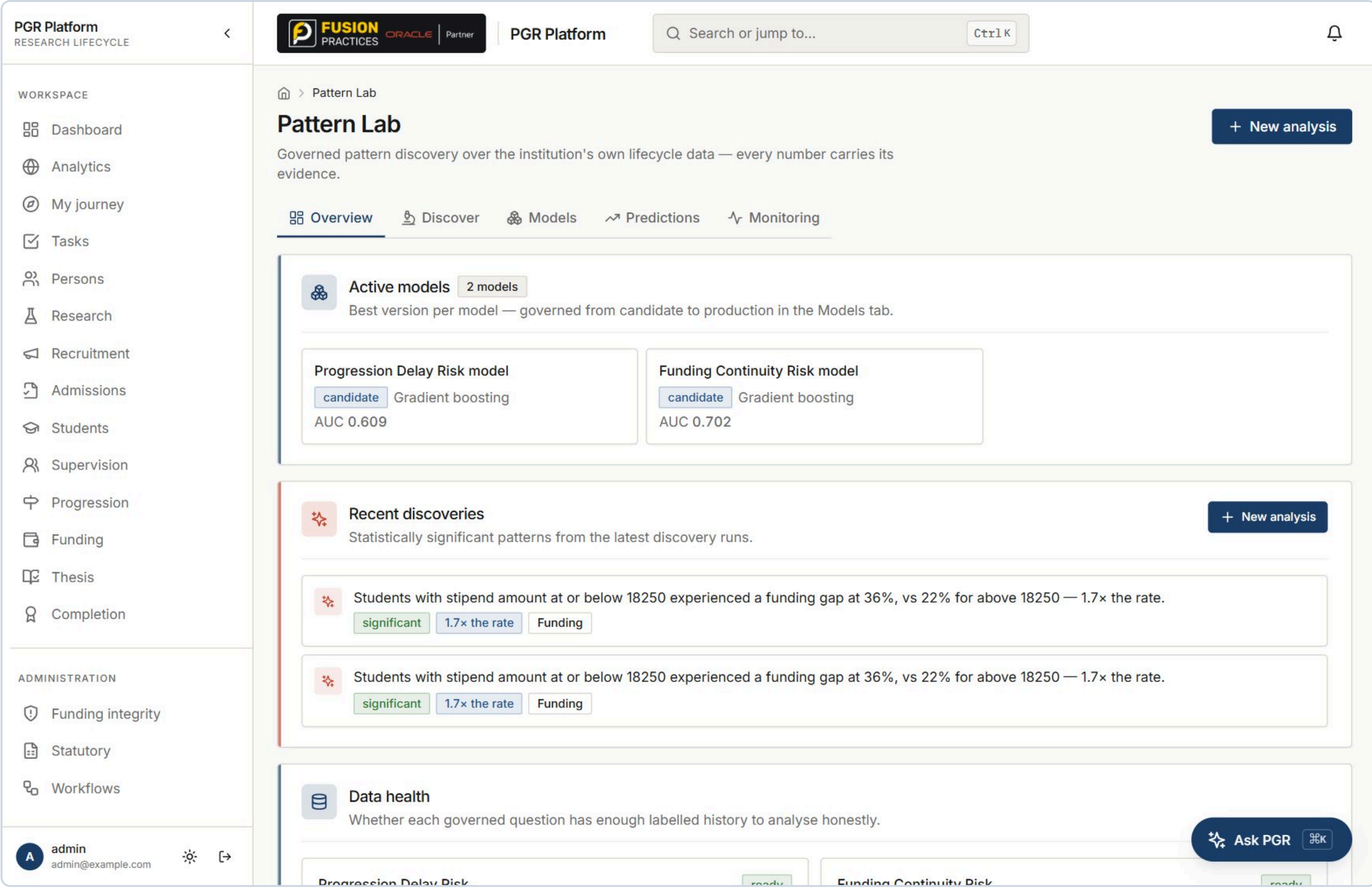
WHAT THIS SCREEN IS

The institutional learning layer. Its home screen is deliberately an administration dashboard rather than a data-science notebook: which models are active, what was recently discovered, and whether each governed question even has enough data to be asked honestly. Five tabs run left to right as a lifecycle: Overview, Discover, Models, Predictions, Monitoring.

WHAT TO DO

1. Open **Advanced** - **Pattern Lab**.
2. Read **Active models**: the best version per model, its algorithm and its AUC.
3. Read one **Recent discovery** out loud in full - it is a complete sentence with both rates and the ratio.
4. Scroll to **Data health** and show two targets ready and two locked.
5. Say the sentence about refusing to model what it cannot model.

> *"Two of these four governed questions are locked, and the platform tells you exactly why - '1 completion recorded; at least 50 needed'. It refuses to model what it cannot model. That refusal is the feature."*



Pattern Lab - the governed ML layer - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Active models	The best version per model, with algorithm and cross-validated AUC.	**2 models** - Progression Delay Risk (gradient boosting, AUC 0.609), Funding Continuity Risk (gradient boosting, AUC 0.702)
Recent discoveries	Statistically significant patterns from the latest discovery runs, each written as a full sentence with both group rates and the ratio.	e.g. "Students with stipend at or below 18250 experienced a funding gap at 36%, vs 22% for above 18250 - 1.7x the rate"
Discovery tags	Each finding carries a significance badge, the effect size, and the domain it belongs to.	`significant` - `1.7x the rate` - `Funding`
Data health - ready	A governed target is analysable when it has at least 50 eligible rows and at least 15 in the minority class.	Progression Delay Risk **ready**, Funding Continuity Risk **ready**
Data health - locked	A locked target states the exact deficiency in plain words.	Completion Forecast: "1 completion recorded; at least 50 needed" - Applicant Outcome: "conversion is ~99.7%; there is almost no negative class to learn from"
Finding count	`ml_finding` rows across all discovery runs.	**22**

THE VALUE IT ADDS

- **A refusal is worth more than a model.** Every ML vendor demonstrates a model. Almost none demonstrate their product declining to build one. In a governance conversation this is the most credible thing on the screen.
- **It self-diagnoses the demo's own weakness.** The Applicant Outcome lock literally reads "conversion is ~99.7%; there is almost no negative class" - the platform noticed the seed artefact you are about to be asked about. Use it: it turns your weakest number into your strongest governance story.
- **Governed targets, not open-ended prediction.** There is no "predict anything" box. Four named questions, each approved, each with a data-health gate. That is what makes it deployable in a university.
- **Discoveries are sentences, not charts.** Both rates and the ratio are in the text, so a finding can be quoted in a committee paper without a screenshot.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"AUC 0.609 and 0.702 - is that good?"	0.5 is a coin flip and 0.70 is a modest but real signal on 322 rows. The platform reports the standard deviation alongside it and its own model card volunteers that the dataset is small. Do not oversell it - the honesty is the product.
"Who approved these four questions?"	They are governed targets defined in the platform. Letting an institution define its own, with an approval step, is a named gap.
"Is the data health live?"	It is computed on demand today. A scheduled refresh is a named gap.

Watch out. The two Recent discovery cards on this screen show the same finding twice - two discovery runs produced the same pattern. If someone notices, say exactly that; do not scroll past it.

Where it goes next. Scheduled data-health refresh; institution-defined governed targets behind an approval step; pin a discovery to a dashboard.

Discovery - evidence, not output admin

`/pattern-lab (Discover)` · used by Institutional research, data governance

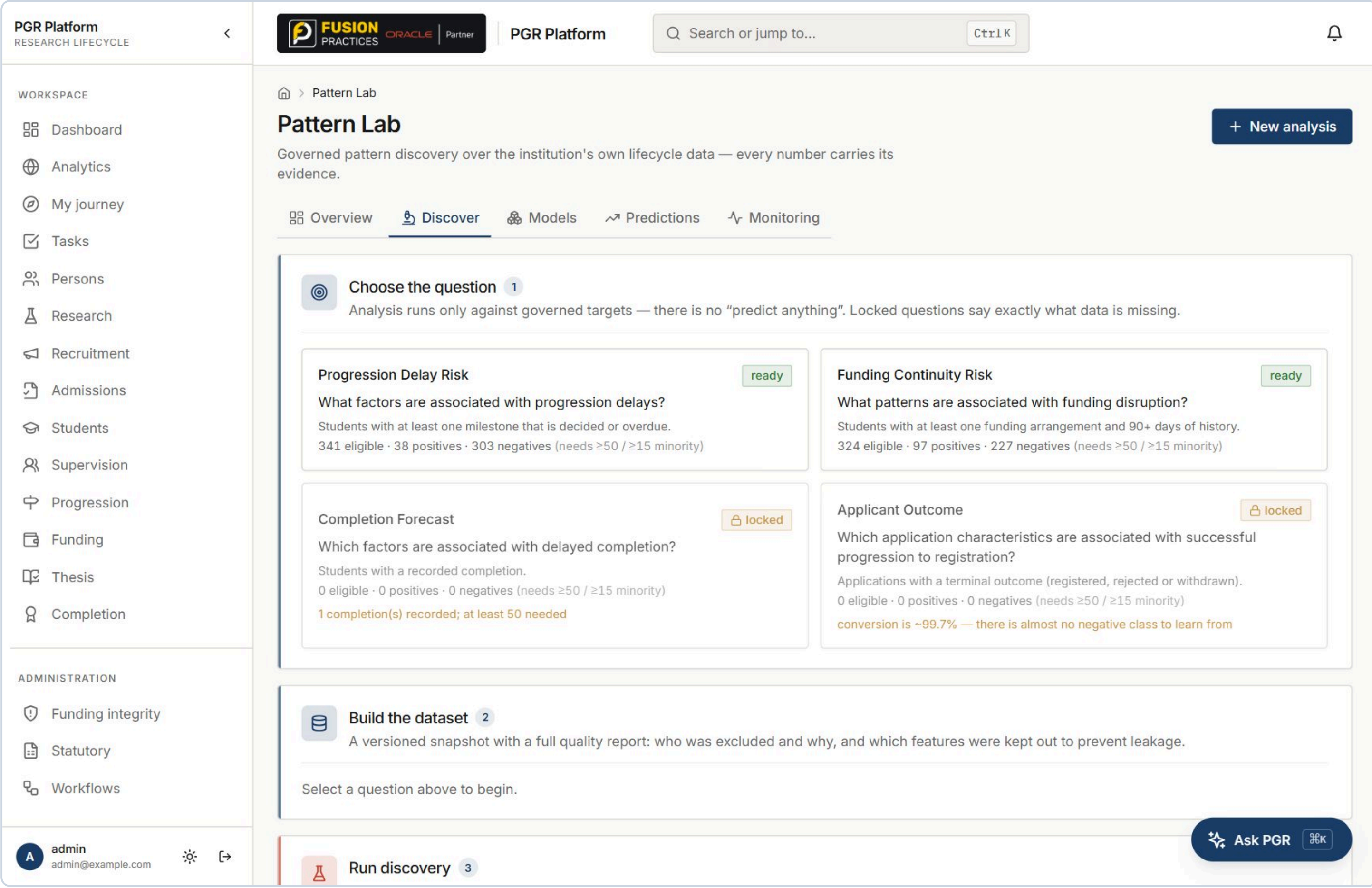
WHAT THIS SCREEN IS

A four-step wizard - choose the question, build the dataset, run discovery, train. The dataset step publishes a full quality report including who was excluded and why, and which features were held out to prevent leakage. That last item is the governance feature most ML tools bury, and here it is a callout.

WHAT TO DO

1. Go to the **Discover** tab.
2. Read the four question cards - two ready, two locked, with the eligibility rule and the counts on each.
3. Pick **Funding Continuity Risk** and build the dataset.
4. Point at the amber **Excluded to prevent leakage** callout and name a held-out feature.
5. Run discovery and open **View evidence** on one finding.

> *"It excluded that feature structurally, because of what it is, not because a reviewer happened to notice. And every finding carries its p-value, its multiple-comparison correction, its confounders and the sentence 'association does not imply causation'."*



Discovery - evidence, not output - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Governed target	Four only. Each states its eligibility rule in plain words on the card.	Progression Delay Risk, Funding Continuity Risk, Completion Forecast, Applicant Outcome
Eligibility rule	Written on the card, e.g. "Students with at least one funding arrangement and 90+ days of history".	Stated before any number is shown
Cohort counts	Eligible, positives, negatives, against the gate 'needs 50 or more, 15 or more minority'.	Funding Continuity: **324 eligible, 97 positives, 227 negatives**. Progression Delay: **341 / 38 / 303**
Locked target	States the exact deficiency rather than greying out.	Completion Forecast: 0 eligible, "1 completion recorded; at least 50 needed"
Dataset snapshot	`ml_dataset` - a versioned snapshot with a quality report: who was excluded and why.	Referenced by hash on every model version, e.g. `ac9a3853...`
Leakage exclusion	Features structurally capable of encoding the outcome are held out at dataset build time and named in the report.	Shown as an amber callout, not a log line
Feature set	Ten features: stipend_amount, meetings_before, supervisors_current, milestones_defined, part_time, project_award_linked, funding_award_linked, supervisor_change, days_to_first_due, arrangements_before / funded_at_cutoff.	Same ten across models, so importances are comparable
Finding evidence	Each finding carries the two group rates, the ratio, a p-value, a multiple-comparison correction, identified confounders and a standing causation disclaimer.	22 findings recorded

THE VALUE IT ADDS

- **Leakage prevention shown, not claimed.** Data leakage is the single most common reason an academic-analytics model is quietly wrong. Naming the held-out features on screen is a claim you can be held to.
- **Statistical honesty by default.** p-value plus correction plus confounders plus a causation disclaimer, on every finding, without anyone opting in. That is the difference between analytics and a dashboard of coincidences.
- **The dataset is versioned and hashed.** Every model version points at the dataset it was built from. Two years later you can say exactly what a model was trained on.
- **The locked cards teach the institution something.** "You cannot answer this question until you have recorded 50 completions" is genuinely useful management information about data maturity.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"Can I ask it anything?"	No, and that is deliberate. Four governed targets. An open prediction box is how analytics projects end up in front of an ethics committee.
"What exactly gets excluded for leakage?"	Have one example ready from the callout before you present. Naming a specific held-out feature is far more convincing than describing the concept.
"Is a p-value enough?"	On its own, no - which is why the finding also carries the correction, the confounders and the disclaimer. The platform is arguing against over-reading its own output.

Watch out. Building the dataset and running discovery take a few seconds each - the latest runs were 29.5 s and 41.9 s end to end for training. Do not run training live in a 15-minute slot; run discovery only, or show pre-built results from the Models tab.

Where it goes next. Pin a finding to a dashboard; scheduled re-discovery as cohorts grow; an institution-defined governed target behind approval.

Models, governance and the model card `admin`

`/pattern-lab (Models)` · used by Data governance, Director of PGR

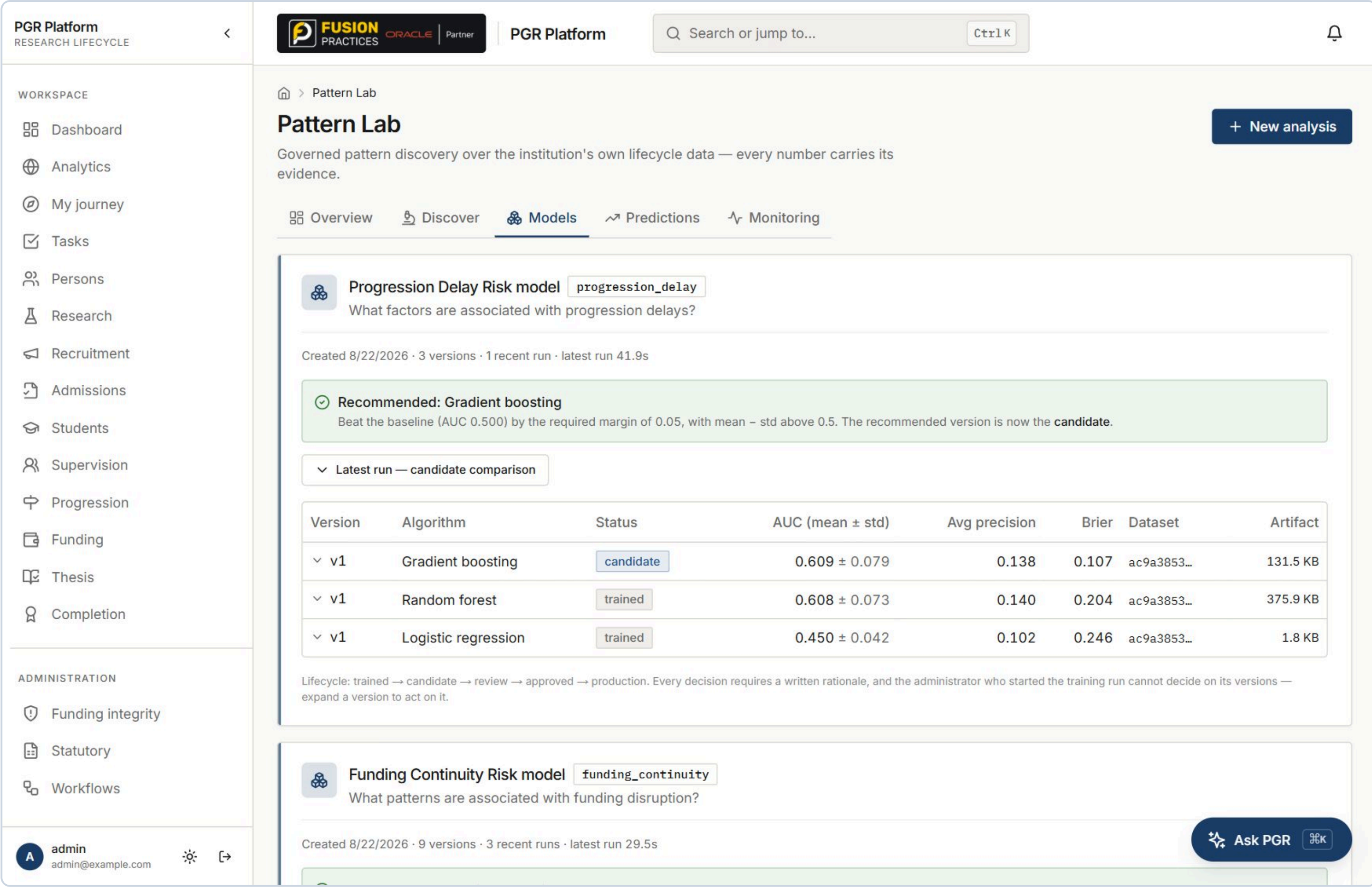
WHAT THIS SCREEN IS

Three algorithms are trained per run and compared against a stated baseline, then a candidate is recommended only if it beats that baseline by a required margin. From there a governed lifecycle runs: trained, candidate, review, approved, production. Every decision needs a written rationale, and whoever started the training run cannot decide on its versions.

WHAT TO DO

1. Go to the **Models** tab.
2. Read the green **Recommended** banner - it names the baseline, the margin and the test.
3. Open **Latest run - candidate comparison** and read the table across: algorithm, status, AUC with standard deviation, average precision, Brier, dataset hash, artifact size.
4. Expand a version to show permutation importance.
5. Open the **Model card** and read the **Limitations** section aloud, verbatim.

> *"The limitations on that card are computed, not written by our marketing team. It volunteers that the dataset is small and the variance is wide. A model that markets itself is a model you cannot trust."*



Models, governance and the model card - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Baseline test	A candidate must beat the stated baseline (AUC 0.500) by a required margin of 0.05, *and* have mean minus standard deviation above 0.5 - so a lucky fold cannot promote a model.	Stated on screen in the recommendation banner
Algorithms compared	Three per run: gradient boosting, random forest, logistic regression. All trained locally with scikit-learn.	Progression Delay: 0.609 / 0.608 / 0.450
AUC (mean +/- std)	5-fold cross-validated area under the ROC curve. The standard deviation is shown next to it, never hidden.	Funding Continuity production v1: **0.702 +/- 0.057**
Average precision	Area under the precision-recall curve - the honest metric when positives are rare.	Funding Continuity **0.523**; Progression Delay **0.138**
Brier score	Mean squared error of the predicted probabilities - lower is better. Measures calibration, not just ranking.	Funding Continuity **0.176**; Progression Delay **0.107**
Precision / recall at 50%	What you would actually get if you acted on everything scored above 0.5.	Funding Continuity: precision **0.695**, recall **0.432**
Training set	Row count and positive count, recorded on the version.	Funding Continuity **n=322, 95 positives**; Progression Delay **n=341, 36 positives**
Permutation importance	Per-feature, computed after training. Negative values mean the feature hurt - and they are shown, not clipped to zero.	Funding Continuity top three: stipend_amount 0.065, meetings_before 0.022, part_time 0.020
Dataset hash and artifact	Each version records the dataset snapshot it was built from and the size of the stored model artifact.	`ac9a3853...` - 131.5 KB gradient boosting, 375.9 KB random forest, 1.8 KB logistic regression
Governed lifecycle	trained - candidate - review - approved - production. Every transition requires a written rationale.	12 versions exist; exactly **1** is in production
Approver separation	The administrator who started a training run cannot decide on its versions. Enforced at the server.	Stated on screen beneath the comparison table

THE VALUE IT ADDS

- **Separation of duties, enforced not documented.** “The person who trained it cannot approve it” is a control an auditor can test in thirty seconds. Most ML platforms have a policy; this has a refusal.
- **A model card that argues against itself.** Computed limitations - small dataset, wide variance - mean the card cannot drift away from the truth as the model ages.
- **Three algorithms and a baseline, every time.** Comparing against a stated baseline with a required margin stops the classic failure where a model barely better than a coin flip reaches production because nobody asked.
- **Only one version can score.** 12 trained versions, exactly one in production. There is never ambiguity about which model produced a number a student is being discussed under.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“These AUCs are not very high.”	Correct, and reported honestly with their standard deviations. On 322 rows of synthetic data that is what you would expect. The governance around the number is what is being demonstrated, not the number.
“Can we bring our own model?”	No. Three algorithms, trained locally, governed the same way. That is deliberate.
“Where does the model run?”	On the institution's own infrastructure, with scikit-learn. Nothing is sent anywhere.
“Can I compare two versions side by side?”	Not yet - side-by-side version comparison and a silent challenger model are named gaps.

> **Watch out.** Negative permutation importances are on screen (e.g. part_time -0.007). If someone asks, that means the feature made the model slightly worse on shuffling - it is noise, and the platform shows it rather than hiding it.

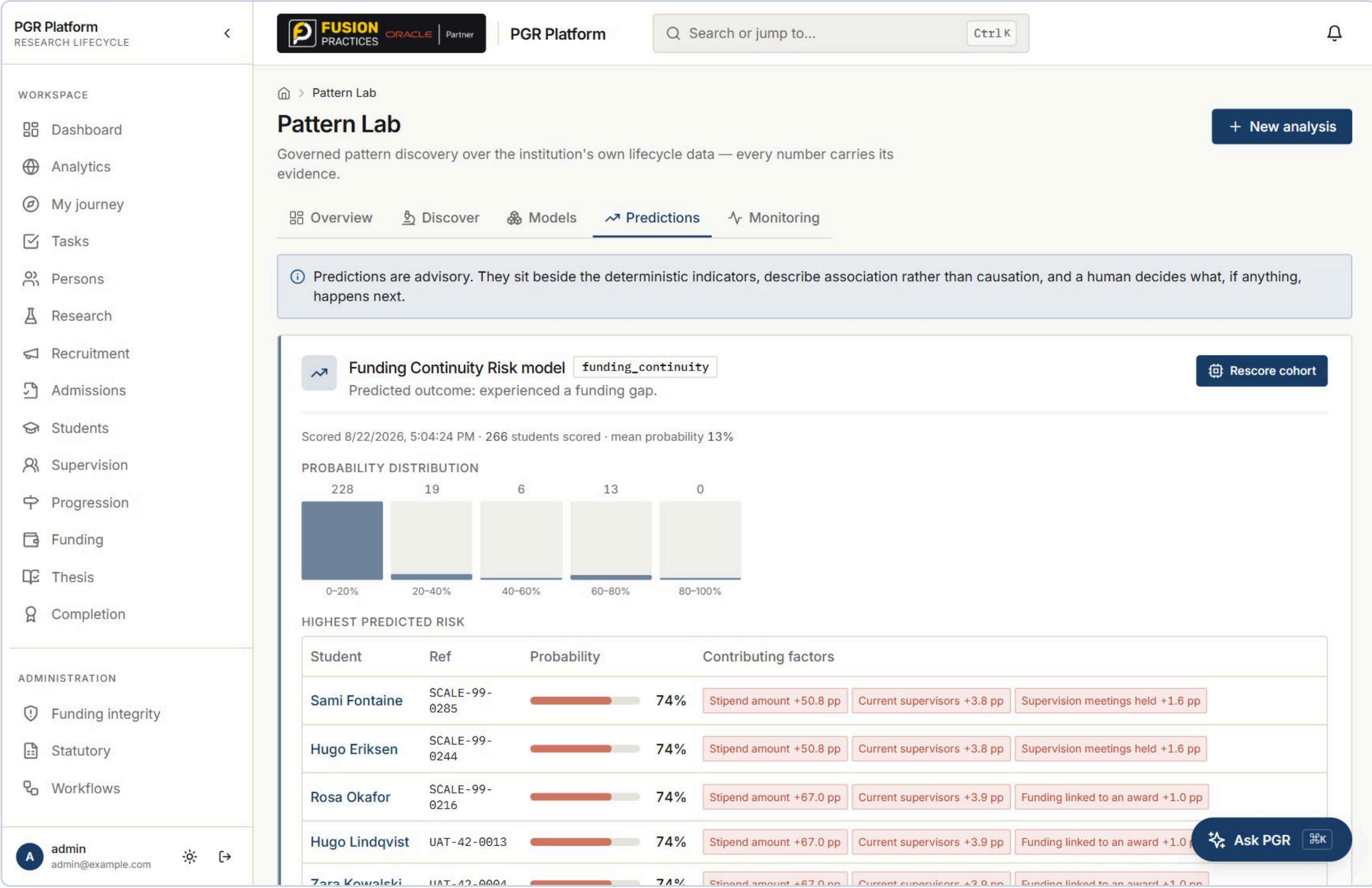
> **Where it goes next.** Side-by-side version comparison; a challenger model scoring silently against production before promotion; automated model-card export for a governance committee.

Predictions with contributing factors `admin`

`/pattern-lab (Predictions)` · used by Director of PGR, Graduate School

WHAT THIS SCREEN IS

Only a version promoted through governance can score. The screen opens on a standing advisory banner - on screen, not in a footnote - then a probability distribution across the cohort and the highest-risk table. Every row's contributing factors are computed for that individual student, in percentage points, not copied from the model's global importances.



WHAT TO DO

1. Go to the **Predictions** tab.
2. Read the blue advisory banner out loud, word for word. Do not paraphrase it.
3. Read the scoring line: when, how many students, mean probability.
4. Show the distribution - the shape is the story, not the top row.
5. Point at two rows whose factor chips differ, and say why they differ.
6. Show **Rescore cohort** and explain it is an explicit act, not a background job.

> *"Those factors are computed for that individual student, not copied from the model. And note the wording at the top - advisory, association not causation, a human decides what happens next. That sentence is on the screen, not in a manual."*

Predictions with contributing factors - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Advisory banner	Permanent, non-dismissible: predictions are advisory, sit beside the deterministic indicators, describe association rather than causation, and a human decides.	Always on screen
Predicted outcome	Stated in plain words, not as a target key.	“experienced a funding gap”
Scored / cohort size / mean	The batch timestamp, how many students were scored, and the mean probability.	Scored 2026-08-22 - **266 students** - mean **13%**
Probability distribution	Five bands of 20 percentage points each, with a count above each bar.	228 / 19 / 6 / 13 / 0 - so **86% of the cohort sits in the lowest band**
Highest predicted risk	Ranked table: student, reference, probability bar, contributing factor chips.	Top of the table is **74%** - Sami Fontaine, Hugo Eriksen, Rosa Okafor, Hugo Lindqvist, Zara Kowalski
Contributing factors	Per student, in percentage points, signed. Two students with the same probability can have different factors and different magnitudes.	e.g. “Stipend amount +50.8 pp” vs “Stipend amount +67.0 pp” on adjacent rows
Prediction store	`ml_prediction` rows, one per student per batch, retained for the monitoring comparison later.	**266** predictions, mean 0.134, max 0.743
Task raising	Off by default. When on, a batch raises a review task for each student at or above the threshold. A task suggests a human look - it never changes a student record.	`pattern_lab.raise_tasks` = **false**; threshold 0.7, tunable 0.5-0.95

THE VALUE IT ADDS

- **The distribution is the trust argument.** 86% of the cohort in the lowest band means the model discriminates rather than flagging everyone. Point at the shape - it is a better answer to ‘is this useful’ than the AUC.
- **Per-student factors make a conversation possible.** “This is flagged mainly because of the stipend level, worth about 50 percentage points” is something a supervisor can act on or dismiss. A bare 74% is not.
- **Advisory by construction, not by policy.** The prediction cannot change a record. The only thing it can do, and only when switched on, is ask a human to look. That is the answer to every ‘is the AI deciding’ question in the room.
- **Scoring is an explicit act.** Rescore cohort is a button. Nobody wakes up to find their students have been silently re-ranked overnight.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“Is the AI making decisions?”	No. Advisory rows, and optionally a review task. Only rules and humans change a student record, and the task switch is off by default. This is the single most important answer in the demo - have it word-perfect.
“Would a student see their score?”	Not in the student portal today. Whether they should is an institutional policy decision, and worth putting back to the room rather than answering for them.
“Five students all at exactly 74%?”	Gradient boosting on a small feature set produces plateaus - students in the same leaf get the same score. Their factor magnitudes still differ, which is visible on the rows.
“How often is it rescored?”	When somebody presses the button. Scheduled scoring is a named gap.

Watch out. The last scoring batch was 2026-08-22 and the monitoring tab now flags this model for review. Do not present these predictions as current - present them as the batch that monitoring is about to grade. That sets up the next screen perfectly.

Where it goes next. Prediction history on the student record; scheduled scoring; a per-institution threshold with its own approval.

Monitoring - closing the loop `admin`

`/pattern-lab (Monitoring)` · used by Data governance, Director of PGR

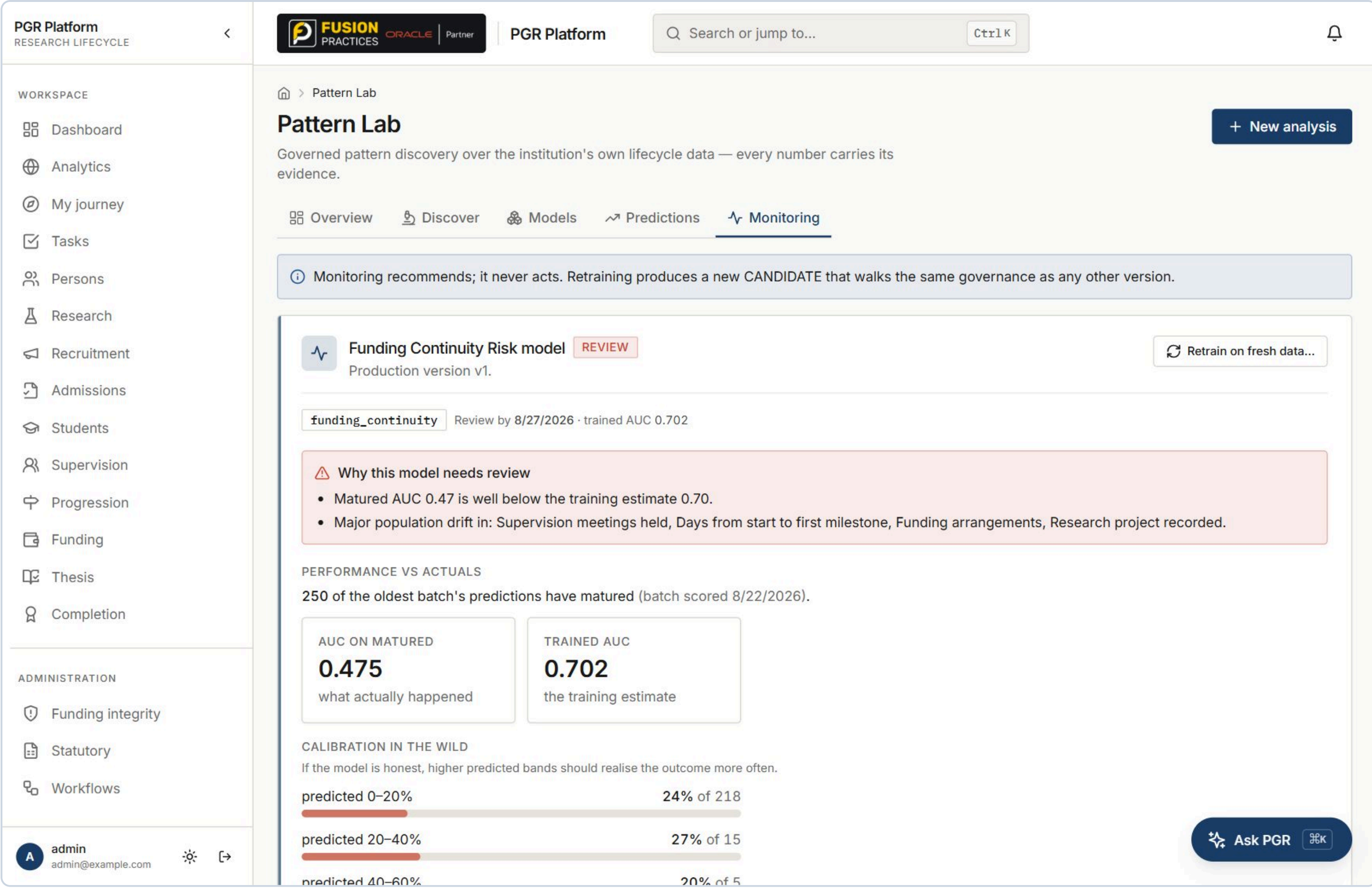
WHAT THIS SCREEN IS

The honest part, and the strongest screen in the whole demo. Matured predictions are compared against what actually happened, population drift is measured per feature, and the health verdict names its reasons in plain words. Retraining produces a candidate that walks the same governance as any other version - monitoring recommends, it never acts.

WHAT TO DO

1. Go to the **Monitoring** tab.
2. Read the banner: monitoring recommends, it never acts.
3. Point at the **REVIEW** badge next to the model name, then read **Why this model needs review** - both bullets, out loud.
4. Show **Performance vs actuals**: matured AUC against trained AUC, side by side.
5. Show **calibration in the wild** and then the drift table.
6. Press nothing. Explain that **Retrain on fresh data** produces a candidate, not a production model.

> *"This model is currently flagged for review by its own monitoring. Its accuracy against what actually happened fell to 0.475 - below a coin flip - against a training estimate of 0.702. The platform told us that. We did not have to notice."*



Monitoring - closing the loop - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Health verdict	A badge on the model - `healthy` or `REVIEW` - with its reasons listed beneath in plain words.	**REVIEW**
Reason: performance	Matured AUC materially below the training estimate.	“Matured AUC 0.47 is well below the training estimate 0.70”
Reason: drift	Named features whose population distribution has moved materially since training.	Supervision meetings held, Days from start to first milestone, Funding arrangements, Research project recorded
Matured predictions	Predictions old enough for the outcome to be known, from the oldest batch.	**250 of the batch** scored 2026-08-22
AUC on matured vs trained AUC	What actually happened, next to what training predicted. Shown side by side, deliberately.	**0.475** actual vs **0.702** trained
Calibration in the wild	Per predicted band, the share that actually realised the outcome. If the model is honest, higher bands should realise more often.	0-20%: 24% of 218 - 20-40%: 27% of 15 - 40-60%: 20% of 5. It is not monotonic, which is the point
Drift table	Per feature, the movement in population distribution since the training snapshot.	Four features flagged as major
Review interval	How long after the latest batch a production model's recommended review date falls - unless drift or performance pulls it to today.	`pattern_lab.review_interval_days` = **90**, tunable 30-365
Retrain on fresh data	Produces a new **candidate**. It walks the same governance path - review, approval by someone other than the trainer, promotion.	Explicit, human-initiated

THE VALUE IT ADDS

- **A model that grades itself in public.** 0.475 against 0.702 is a bad number, on screen, unprompted. Showing a room the failure state of your own ML is the single most credible thing in this demo. Do not skip it, and do not apologise for it.
- **Calibration, not just accuracy.** “When it says 60%, does 60% of that group actually experience it?” is the question that matters when a human is going to act on the number. Most tools never ask it.
- **Drift is named per feature.** “Supervision meetings held has drifted” is actionable - it usually means a process changed, which is management information in its own right.
- **The loop closes without automation.** Monitoring recommends, a human retrains, a different human approves. There is no path from a bad metric to a silently swapped production model.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“Your model does not work.”	On this synthetic cohort, on this batch, correct - and the platform is the thing telling you. Agree immediately. The alternative product is one where the same thing is true and nobody finds out.
“Why did it drift so fast?”	Because the demo data was generated, then more of it was generated. In a real deployment drift over months is normal and the review interval is 90 days.
“Can it retrain automatically?”	It can retrain on demand, and the result is a candidate. Scheduled monitoring with alerting and automatic challenger retraining - still requiring human promotion - are named gaps.

Watch out. Do not click Retrain during a demo. It takes ~30 s, produces a candidate you then have to explain, and it moves the model list you already showed. Explain the button instead.

Where it goes next. Scheduled monitoring with alerting; automatic challenger retraining that still requires human promotion; drift alerts routed to the data governance owner.

Institution policy - settings that do something `admin`

`/settings (Institution policy)` · used by Institution Administrator only

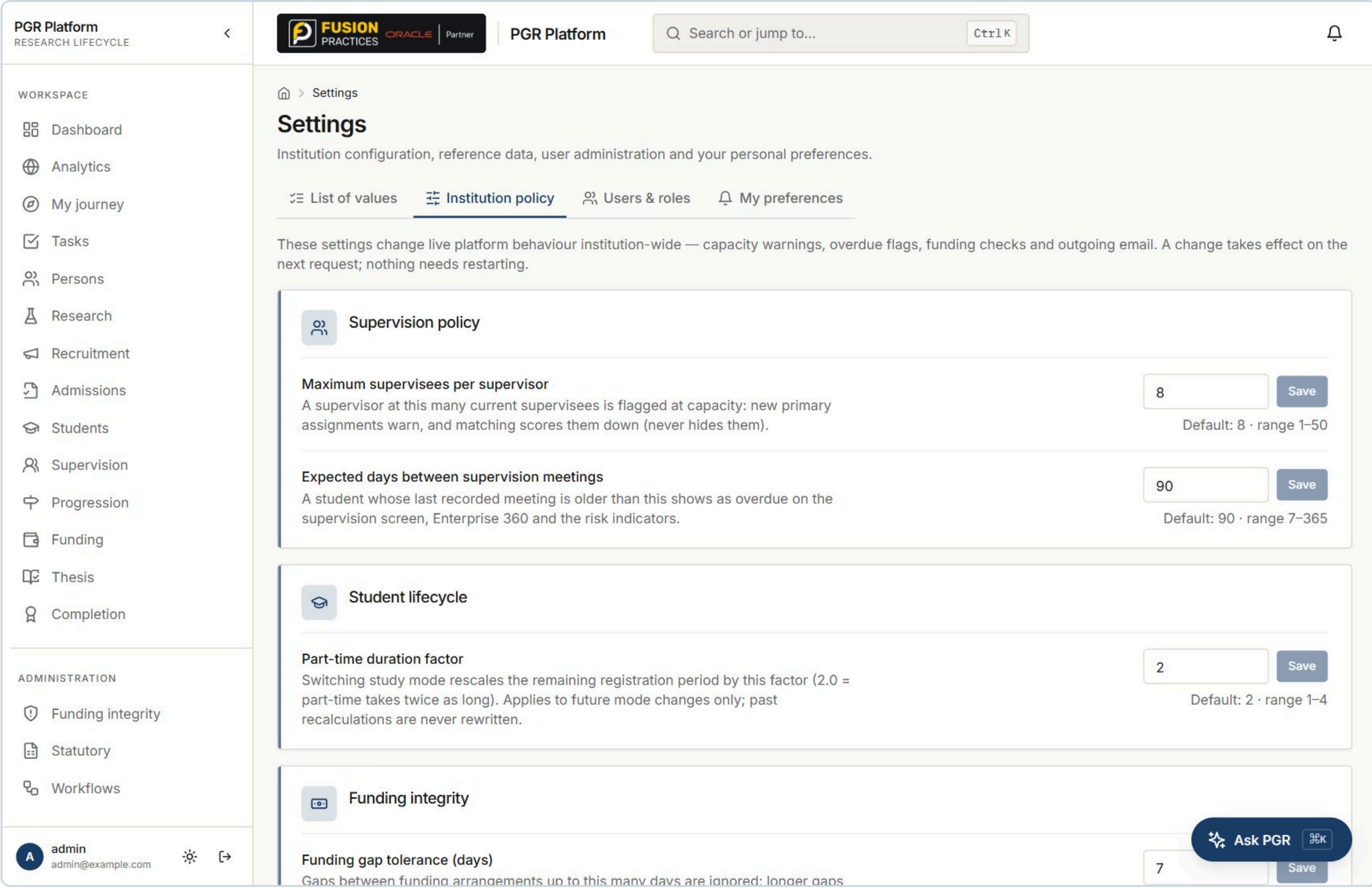
WHAT THIS SCREEN IS

Configuration is not decorative here. Every setting is defined once in a registry that the API, the validation and this screen all read, so there is exactly one place a setting exists. Each one is wired into live behaviour - change a number and the platform behaves differently on the next request, with no deployment and no restart.

WHAT TO DO

1. Open **Settings** - **Institution policy**.
2. Read the line at the top: a change takes effect on the next request; nothing restarts.
3. Show **Maximum supervisees per supervisor** with its default and its range.
4. Name the three places that one number is wired into: the assignment guard, the matching score, the capacity flag.
5. Scroll to the Pattern Lab section and show the task-raising switch, off by default.

> *"Change that from 8 to 2 and the platform starts warning on the next supervisor assignment immediately - no deployment, no restart. There is a test that proves exactly that."*



Institution policy - settings that do something - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Maximum supervisees per supervisor	Wired into three behaviours: new primary assignments warn at capacity, supervisor matching scores a full supervisor down (never hides them), and the capacity flag on the person record.	**8** - default 8, range 1-50
Expected days between supervision meetings	Drives "meetings overdue" on the supervision caseload, on Enterprise 360 and in the risk indicators.	**90** - default 90, range 7-365
Part-time duration factor	A mode change rescales the remaining registration period by this factor. Future mode changes only; past recalculations are never rewritten.	**2.0** - default 2.0, range 1.0-4.0
Funding gap tolerance (days)	Gaps up to this are ignored; longer gaps raise a `funding_gap` finding in both the per-student lineage and the cohort integrity check.	**7** - default 7, range 0-90
Send email notifications	Institution-wide switch. Off means notifications are still recorded in the bell menu but no email leaves the platform. Overrides every personal preference.	**true**
Email sender name	The display name on outgoing mail. The SMTP relay itself stays in the server environment, never in a table an API can read back.	**PGR Platform**
Raise tasks from high predictions	Off: predictions are display-only. On: a scoring batch raises a review task per student at or above the threshold. A task never changes a record.	**false** - off by default
Task-raising probability threshold	Minimum predicted probability before a review task is raised.	**0.70** - range 0.5-0.95
Model review interval (days)	How far after the latest batch a production model's review date falls, absent drift or performance signals.	**90** - range 30-365
Allow LLM fallback in Ask PGR	Off: the assistant answers only from its deterministic parser and concept graph. On: unrecognised questions may go to a configured LLM. Requires a key in the environment either way.	**false** - off by default
What is deliberately not here	SMTP credentials and the database URL stay in the server environment. Enum value sets are domain vocabulary with code attached and are exposed read-only.	Stated in the registry itself

THE VALUE IT ADDS

- **Institutional policy without a change request.** Supervision load caps, meeting cadence and funding tolerance are exactly the numbers that differ between institutions and change after a review. Every one of them is a text box here.
- **One definition, one place.** The registry references the shipped constants rather than copying the numbers, so the code and the settings screen can never disagree about a default. That is why the defaults on screen are trustworthy.
- **Validation is part of the definition.** Every setting carries a type and a range, enforced server-side. You cannot set the supervision cap to zero and break assignment.
- **Two safety switches, both off by default.** Pattern Lab task raising and the assistant's LLM fallback both default to off. Nothing consequential and nothing leaves the building unless somebody deliberately turns it on.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"Can I see who changed a setting and when?"	The change is audited as a state-changing request, but there is no settings-history view. That is a named gap and an easy one.
"Can different departments have different caps?"	Not yet - settings are institution-wide. Per-department overrides are a named gap.
"Where are the SMTP credentials?"	In the server environment, deliberately not in any table an API can read. Say this - it is the answer a security reviewer is listening for.

Watch out. If you promise to change a value live, change it back before the next screen. The 8 drives the matching score you may already have shown.

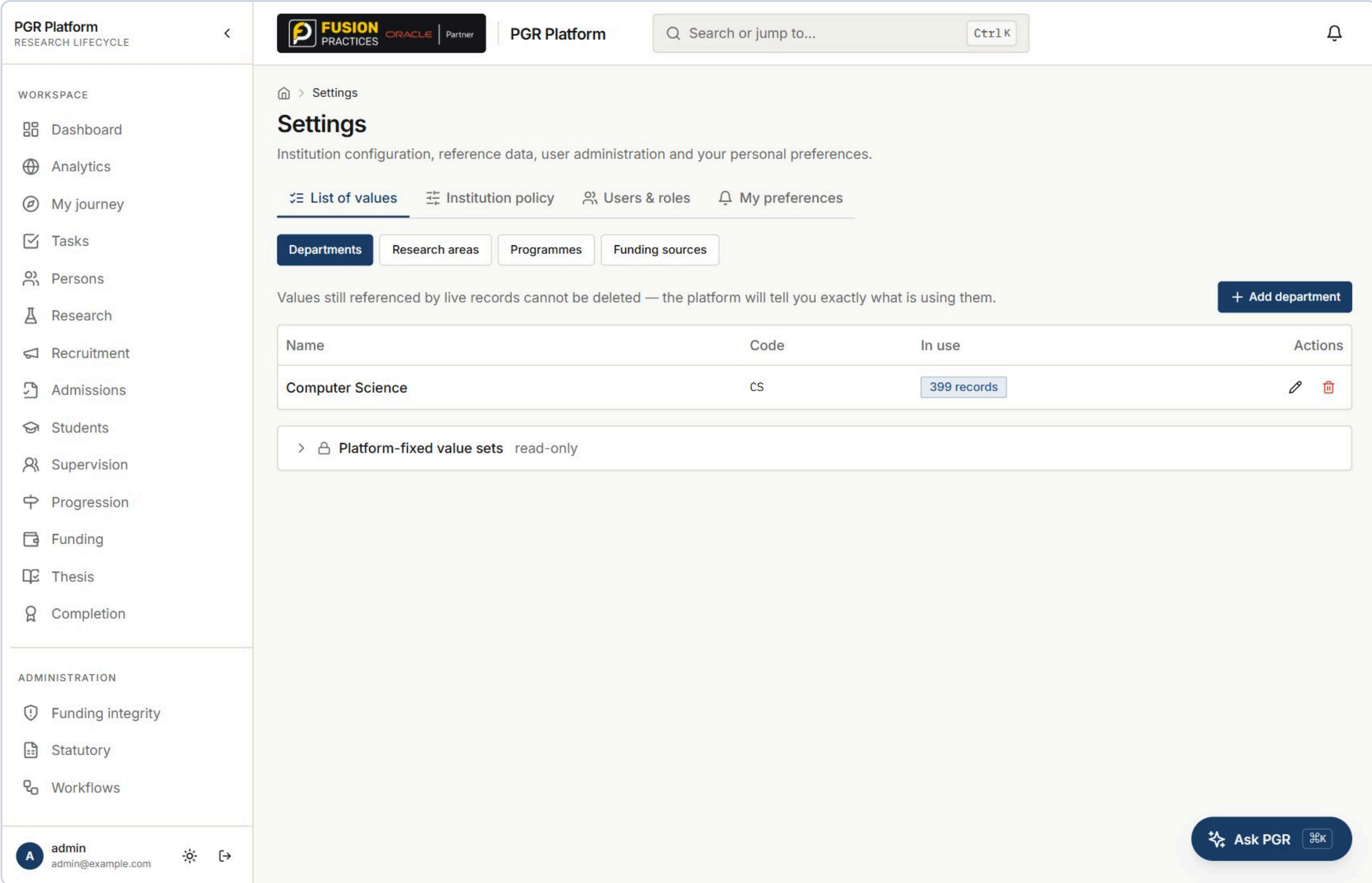
Where it goes next. A settings-history view showing changes over time; per-department overrides; an approval step for the two safety switches.

Reference data with in-use protection `admin`

`/settings (List of values)` · used by Institution Administrator

WHAT THIS SCREEN IS

Departments, research areas, programmes and funding sources are administrator-managed. Every row shows how many live records point at it, and deleting one that is in use is refused with an exact account of what is using it. Below the editable lists sits a locked panel of platform-fixed value sets, exposed read-only.



WHAT TO DO

1. Open **Settings** - **List of values**.
2. Move across the four tabs: Departments, Research areas, Programmes, Funding sources.
3. Point at the **In use** count on a row.
4. Try to delete **Computer Science** and read the refusal.
5. Expand **Platform-fixed value sets** and explain why those are read-only.

> *"It will not let you delete Computer Science because 399 records point at it - and it tells you that, instead of failing silently or orphaning three hundred and ninety-nine students."*

Reference data with in-use protection - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Departments	Administrator-managed, with a code and a live in-use count.	**1** - Computer Science (CS), **399 records** in use
Research areas	Administrator-managed. Used by supervisor matching and by the student record.	**6**
Programmes	Administrator-managed. Each carries its own milestone definitions.	**1** - PhD Computer Science
Funding sources	Administrator-managed, typed as research council / university / external / self-funded.	**2** - UKRI EPSRC, University Scholarship Fund
In-use count	A live count of records referencing this value, computed at display time.	Shown as a chip on every row
Delete protection	Refused at the server when the count is non-zero, and the refusal names what is using it.	The demo case is 399 records against Computer Science
Platform-fixed value sets	Enum vocabularies - student status, funding type, milestone status and so on - each with code attached to the value. Exposed read-only via `/reference/value-sets` .	Renaming one from a settings screen would orphan the logic behind it, so it is not offered

THE VALUE IT ADDS

- **Orphaned data becomes impossible.** The classic student-system failure is a reference value deleted while records still point at it, producing blanks in a statutory return eighteen months later. This makes it unreachable.
- **The refusal is informative, not just a block.** Naming 399 records tells the administrator what to do next. A bare 'cannot delete' tells them nothing and gets escalated to support.
- **It draws the line between vocabulary and configuration.** Departments are yours to manage. Student status is not, because code depends on each value. Being explicit about that distinction is what stops a configuration screen becoming a way to break the platform.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"What if we reorganise departments?"	Merge-and-retire is a named gap - today you can add and rename but not merge. It is the first thing a real institution will hit and it is worth saying before they do.
"Only one department and one programme?"	Seed data. The model is fully multi-department and multi-programme.
"Can we add our own value sets?"	Not for the platform-fixed vocabularies. The four administrator-managed lists are extensible.

Watch out. With one department and one programme this screen looks trivial. Its entire value is the delete refusal - go straight to it. Do not spend time on the lists themselves.

Where it goes next. Merge-and-retire for reference values so a department can be reorganised without breaking history; effective-dated reference values.

Users, roles and the invitation flow `admin`

`/settings (Users & roles)` · used by Institution Administrator

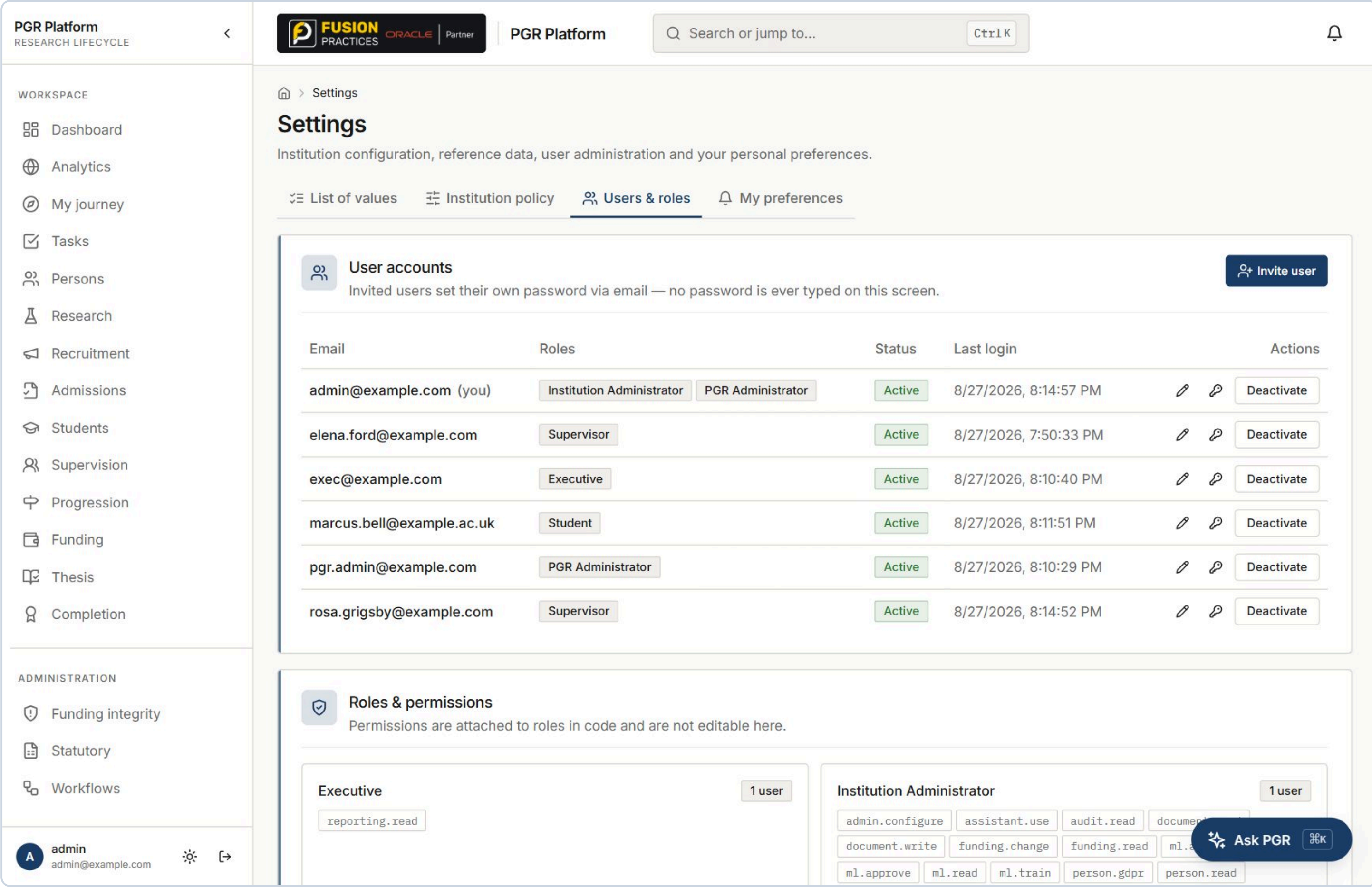
WHAT THIS SCREEN IS

Five roles, 23 permissions, and an invitation flow in which no password ever passes through an administrator - inviting a user sends them a link to set their own. Self-lockout is prevented at the server: you cannot deactivate yourself or strip your own administrator access.

WHAT TO DO

1. Open **Settings** - **Users & roles**.
2. Show the five roles and the permissions each carries.
3. Walk the invitation flow and say the sentence about passwords.
4. Try to deactivate your own account and read the refusal.
5. If the room is security-minded, mention the lockout and revocation behaviour.

> *"You cannot deactivate yourself or remove your own administrator access. Recovering from that would need database surgery, so the platform simply refuses to let you do it."*



Users, roles and the invitation flow - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Roles	Five, each a named bundle of permissions.	Institution Administrator, PGR Administrator, Supervisor, Executive, Student
Permissions	Fine-grained and fail-closed: an unknown or missing permission denies.	**23** permissions across the platform
Users	Accounts, each with one or more roles.	**6** demo accounts
Invitation flow	An invited user receives a link and sets their own password. No password is ever typed, seen or stored by an administrator.	Real SMTP; bounces recorded in `email_bounce`
Password storage	pbkdf2_sha256. Never stored or logged in recoverable form.	Criterion `S4`
Failed-login lockout	Five failed attempts lock the account; a correct password afterwards still fails.	Criterion `S5`, tested
Logout revocation	Logging out genuinely revokes - a stolen refresh token stops working.	Criterion `S6`, tested
Self-lockout prevention	Server-side refusal to deactivate your own account or remove your own administrator role.	Refusal shown live
Authorisation cost	Permissions are embedded as JWT claims, so authorisation adds no database round-trip per request.	Criterion `P4`

THE VALUE IT ADDS

- **Administrators never handle passwords.** The most common credential-handling failure in university systems is an administrator setting a password and emailing it. The flow here makes that impossible rather than discouraged.
- **Fail-closed authorisation.** An unknown permission denies. New endpoints are secure by default rather than open by accident - which is the failure mode that actually happens.
- **The platform protects itself from its own administrators.** Self-lockout prevention is a small feature that says a great deal about how the rest was built.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“We use SSO.”	The configuration hooks exist; SSO against the institution's identity provider is the number one roadmap item and is genuinely small. Say it is not done.
“Can we build our own roles?”	Not yet. A role designer letting an institution compose roles from the 23 permissions is a named gap.
“Are there access reviews?”	No scheduled access review exists. Worth capturing as a requirement if the room raises it.
“Is there rate limiting on login?”	Account lockout after five failures, yes. Endpoint rate limiting, no - it is marked not implemented in the non-functional criteria.

Watch out. Do not actually deactivate anything other than your own account for the refusal demo. Six accounts exist and you need all of them for the rest of the script.

Where it goes next. SSO via the institution's identity provider; a role designer over the 23 permissions; scheduled access reviews; endpoint rate limiting.

Permission boundary in action `pgr`

`/settings` · used by PGR Administrator - the operational role

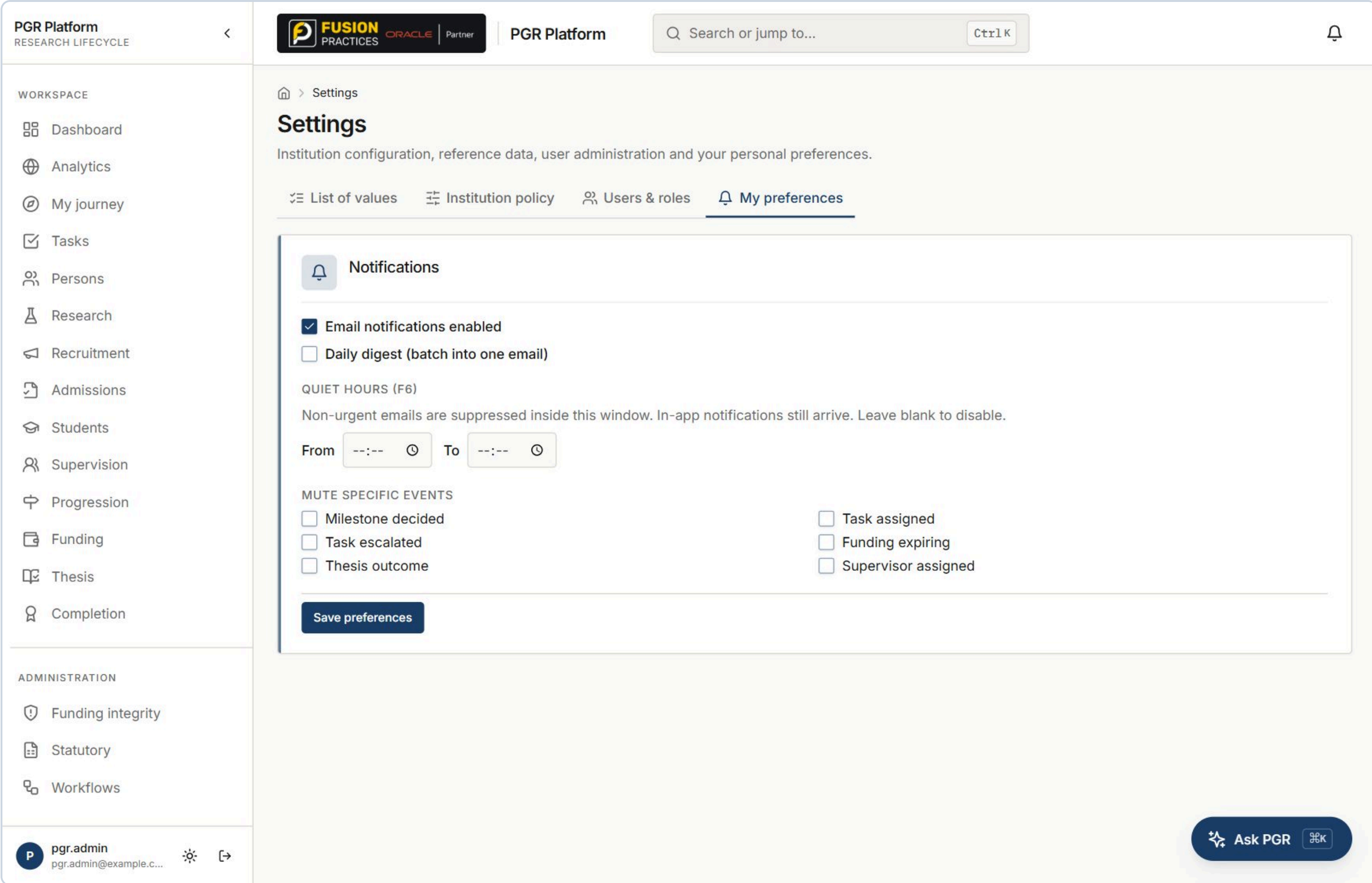
WHAT THIS SCREEN IS

The most convincing way to demonstrate a permission model is to be refused by it, live. The PGR Administrator runs the entire operation - recruitment through statutory returns - and cannot change institution policy, cannot manage users and cannot approve a model. That separation is deliberate and it is enforced at the server.

WHAT TO DO

1. Sign in as **pgr.admin@example.com** / `pgr123` .
2. Open **Settings**. You land on **My preferences** - the tab this role owns.
3. Show what this role does own: notification preferences, quiet hours, per-event mutes.
4. Now click into **Institution policy** and let the server refuse. Read the message.
5. Say the sentence about separation being deliberate.

> *"This is a PGR Administrator. They run the entire operation - recruitment through statutory returns. They cannot change institution policy and they cannot approve a model. That separation is deliberate, and it is the server enforcing it."*



Permission boundary in action - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
What this role owns	My preferences: email on/off, daily digest, quiet hours, and six per-event mutes.	Milestone decided, Task assigned, Task escalated, Funding expiring, Thesis outcome, Supervisor assigned
Quiet hours	Non-urgent emails suppressed inside a window. In-app notifications still arrive. Blank disables it.	Per user, unset by default
What this role does not have	Institution policy, reference data management, user administration, model approval.	Enforced by permission, checked server-side on every request
Fail-closed	A missing permission denies rather than defaults to allow.	Criterion `S2` , tested
Out-of-scope reads	Return 404, not 403 - so the existence of a record is not leaked to someone who should not see it.	Criterion `S3` , tested

THE VALUE IT ADDS

- **Separation of duties you can watch fail.** Operational power and configuration power are different powers. Demonstrating the boundary by being refused is worth more than any diagram of the role model.
- **The operational role is genuinely powerful.** Make this point too - a PGR Administrator can run the whole lifecycle. This is not a cut-down role, it is a differently-bounded one.
- **Personal notification control is a retention feature.** Quiet hours and per-event mutes are why busy administrators keep notifications turned on.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“The tabs are still visible.”	Correct - and answer it before they say it. The navigation is not permission-filtered in this build; the refusal happens at the server when the tab is opened. Argue that this is the stronger demonstration anyway: hiding a menu proves nothing, being refused proves everything.
“Can we change what this role can do?”	Not from the UI. A role designer over the 23 permissions is a named gap.

> **Watch out. The v1 script was wrong here.** It claimed institution policy, reference data and users are ‘not available’ to this role. The tabs render. Click into one and show the server refusing - and check this behaviour yourself before the demo so you know exactly what the refusal looks like.

> **Where it goes next.** Permission-filtered navigation so the menu matches the entitlement; a role designer over the permission list.

The student's own view - concurrent identities `student`

`/portal` · used by The researcher

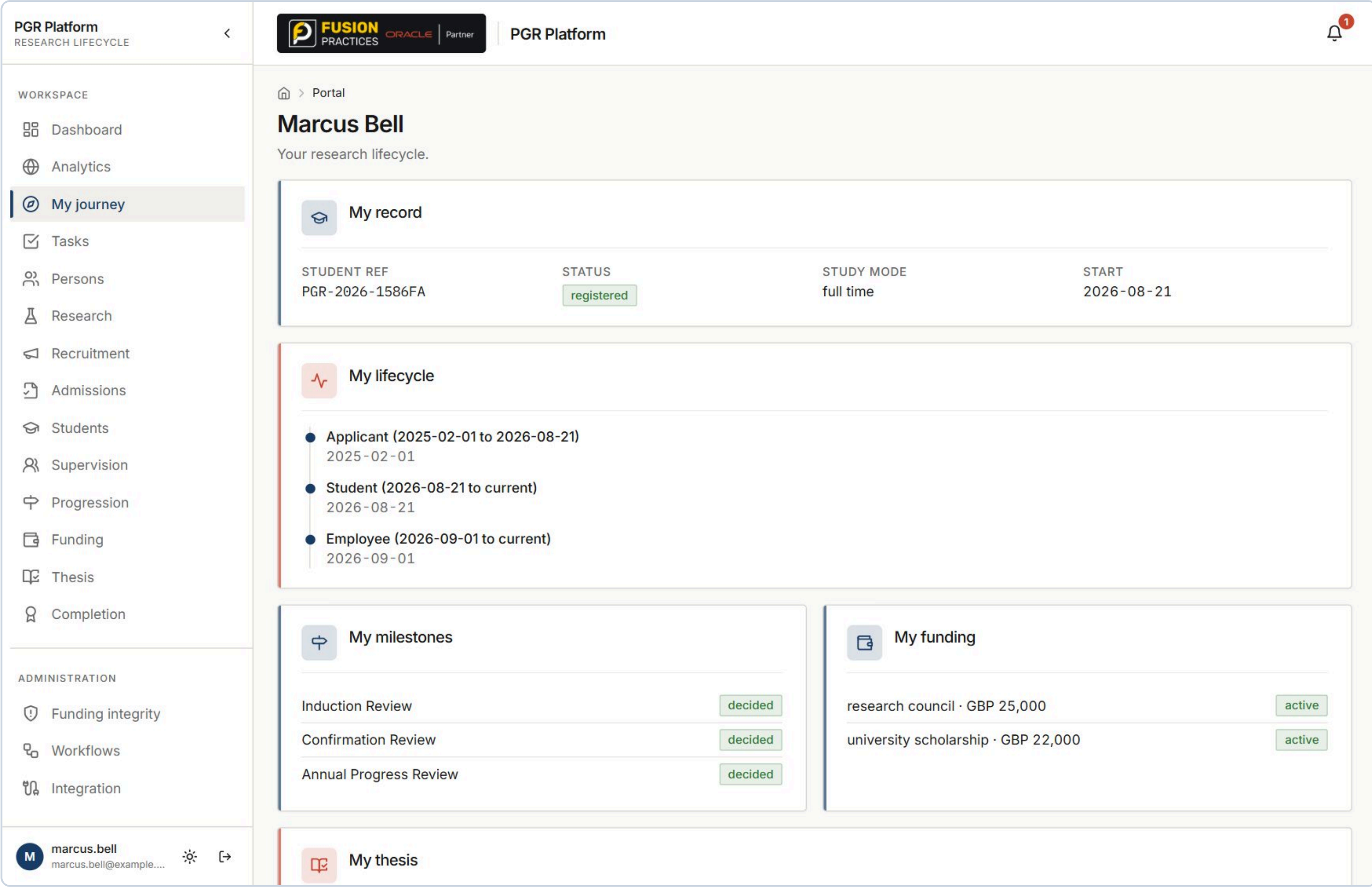
WHAT THIS SCREEN IS

Closes the loop: the same records, from the student's side. Record, lifecycle, milestones, funding and thesis, scoped to them alone. This screen also proves the platform's signature claim - one person holding two live identities at the same time, with overlapping date ranges, on one person record.

WHAT TO DO

1. Sign in as **marcus.bell@example.ac.uk** / `student123` .
2. Open **My journey**.
3. Read **My lifecycle** aloud with the dates - and stop on the overlap.
4. Show **My milestones** and **My funding** - two active arrangements, blended.
5. Say the sentence about two systems and a reconciliation process.

> *"Marcus is a registered PGR student and a member of staff at the same time - look at the dates, they overlap. One person record, two live identities. Most systems would need two accounts and a reconciliation process between them."*



The student's own view - concurrent identities - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
My record	Reference, status, study mode, start date - the same `student` row an administrator sees, scoped to this person.	PGR-2026-1586FA - registered - full time - start 2026-08-21
My lifecycle	`person_relationship` rows with their date ranges. Concurrent identities are the design, not an edge case.	Applicant 2025-02-01 to 2026-08-21 - Student 2026-08-21 to current - **Employee 2026-09-01 to current**
Population-wide identities	The same structure across everyone.	344 student, 63 employee, 2 applicant, 2 alumni relationships over 410 persons
My milestones	The student's own instantiated milestones and their decision state.	Induction Review, Confirmation Review, Annual Progress Review - all **decided**
My funding	The student's own arrangements with type, amount and state. Blended funding is normal.	research council GBP 25,000 **active** + university scholarship GBP 22,000 **active**
My thesis	Thesis status through preparation, submitted, under examination, corrections, approved.	Cohort-wide: 26 preparation, 29 submitted, 39 under examination, 31 corrections, 31 approved
Scope	The portal returns only this person's records, enforced server-side.	The same row-scoping mechanism as the supervisor caseload

THE VALUE IT ADDS

- **The identity claim, demonstrated rather than described.** PGR students who are also teaching assistants, research assistants or technicians are the norm, not the exception. Every institution runs a reconciliation between the student system and HR to cope. Here the two identities are two dated rows on one person.
- **Transparency reduces administrative load.** Most PGR office email is a student asking what their status, end date or funding is. This answers it without a person.
- **It proves the scoping model works in both directions.** The same mechanism that gives a supervisor 13 students gives a student exactly one record - theirs.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
“Can students upload evidence or confirm meetings?”	Not yet - both are named gaps and both are obvious. The portal is read-only today.
“Is it mobile-friendly?”	Not specifically. A mobile layout is a named gap.
“Does the student see their risk prediction?”	No. Whether they should is an institutional policy question - put it back to the room.

Watch out. Do not open My journey as admin. The admin account has no person record and the screen shows an empty state. Marcus Bell is the account with the overlapping identities - use that one, and land on the lifecycle panel deliberately.

Where it goes next. Milestone evidence upload; supervision meeting confirmation from the student side; a mobile-friendly layout.

Executive - numbers without records `exec`

`/dashboard` · used by Pro-Vice-Chancellor, Dean, board

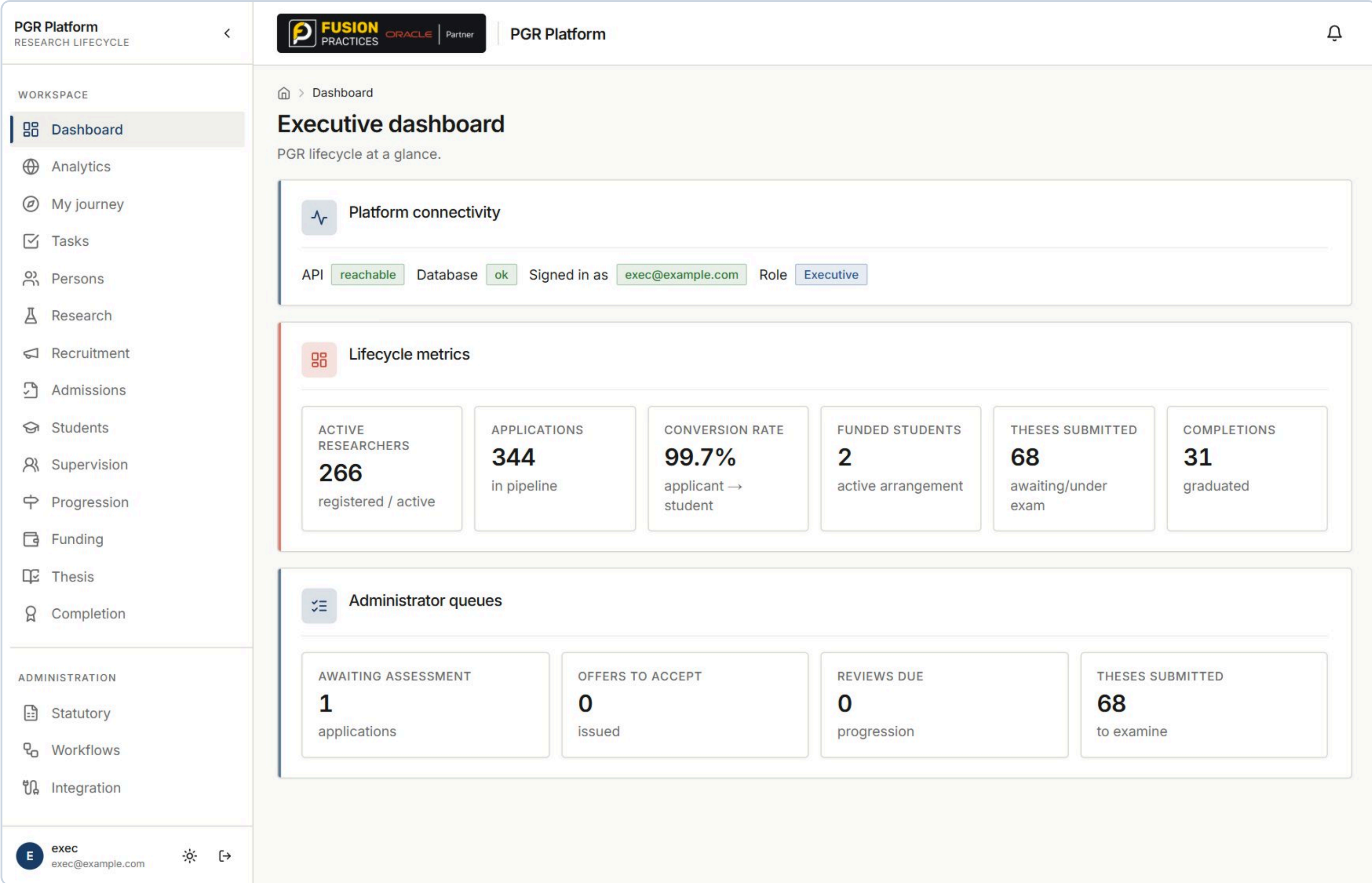
WHAT THIS SCREEN IS

The cleanest single permission story in the platform. One permission - dashboards - and individual student records are refused by the server. The Executive sees the same tiles as the administrator, computed by the same queries, so the board pack and the operational system can never disagree.

WHAT TO DO

1. Sign in as **exec@example.com** / `exec123`.
2. Show the dashboard is fully available - same tiles, same numbers.
3. Point at the role chip: a single role, **Executive**.
4. Now click **Students** in the sidebar and let the server refuse. That refusal is the demonstration.
5. Say the sentence about the server rather than the menu.

> *"A Pro-Vice-Chancellor gets the headline numbers and is refused individual student records - by the server, not by hiding a menu item. And these are the same numbers the registry is working from, because it is the same query."*



Executive - numbers without records - every number on the screen

COMPONENTS AND HOW EACH IS COMPUTED

COMPONENT	HOW IT IS COMPUTED	VALUE TODAY
Role	A single role with dashboard permissions only.	**Executive** - contrast with the admin account's two roles
Lifecycle metrics	Identical to the administrator view, computed by the same service.	266 / 344 / 99.7% / 2 / 68 / 31 - byte-for-byte the same
Administrator queues	Also visible. Executives see what is waiting on the office, without seeing who it is about.	1 / 0 / 0 / 68
Student records	Refused. Row-level and route-level permission, enforced server-side.	The refusal is the demonstration
Personal data in URLs	Never - ids only, no personal data in query strings.	Criterion `D3`

THE VALUE IT ADDS

- **One number, two audiences.** The board and the registry read the same figure from the same query. Institutions routinely discover mid-committee that the board pack and the operational system disagree, and there is no way to find out which is right.
- **Aggregate access without personal data access.** This is exactly the separation an information governance review asks for, and it is the cheapest possible answer to 'who can see student data'.
- **No separate executive reporting build.** No cube, no extract, no BI licence, no refresh window - and therefore no second thing to maintain and reconcile.

OBJECTIONS AND HOW TO ANSWER THEM

THEY ASK	YOU SAY
"The menu still shows Students."	**Say it first.** The navigation is not permission-filtered in this build. Click it and let the server refuse. That is a stronger demonstration than a hidden menu item, which proves nothing about the API.
"Can they drill into a department?"	Not yet - a department scope is a named gap and the obvious next step for this role.
"Can they get this monthly by email?"	No. Scheduled board-ready PDF summaries are a named gap.

Watch out. The v1 script was wrong here. It said there is 'no route to individual student records'. The sidebar still lists Students, Persons, Supervision and Funding for this account. Do not claim the route is absent - claim, and then show, that it is refused.

Where it goes next. Permission-filtered navigation; a department scope so a Dean sees only their faculty; board-ready scheduled PDF summaries emailed monthly.

Appendix - the governance model in one page

WHO CAN DO WHAT

ROLE	BOUNDARY
Institution Administrator	Everything, including institution policy, reference data, users and model approval.
PGR Administrator	The whole operation - recruitment through statutory returns. No institution policy, no user administration, no model approval.
Supervisor	Their own caseload only, enforced in the query. Another supervisor's student returns 404, not 403.
Executive	Dashboards. Individual student records refused at the server.
Student	Their own record only, through the portal.

THE CONTROLS THAT ARE ENFORCED, NOT DOCUMENTED

- **Fail-closed authorisation.** An unknown or missing permission denies.
- **404 not 403** on out-of-scope reads, so existence is not leaked.
- **Approver separation** - whoever started a training run cannot decide on its versions.
- **Requester and approver recorded separately** for suspensions and extensions.
- **Self-lockout prevented** at the server.
- **Reference values in use cannot be deleted**, and the refusal names what is using them.
- **Signed-off statutory profiles are immutable** until unsigned, and cannot be signed off with unmapped mandatory fields.
- **Audit written in middleware**, so no route author can skip it, and an audit failure never fails the user's request.
- **Secrets stay in the environment** - SMTP credentials and the database URL are never in a table an API can read.

THE TWO AI SAFETY SWITCHES - BOTH OFF BY DEFAULT

SWITCH	OFF MEANS
`pattern_lab.raise_tasks`	Predictions are display-only. On, a scoring batch raises a review task per student above the threshold - a task suggests a human look, it never changes a record.
`assistant.llm_enabled`	The assistant answers only from its on-premise deterministic parser and concept graph. Nothing leaves the institution. On, unrecognised questions may go to a configured LLM - and it still needs a key in the server environment.

THE FIVE ANSWERS TO HAVE WORD-PERFECT

“Is the AI making decisions?” No. Predictions are advisory rows and, optionally, review tasks. Only rules and humans change a student record - and the task-raising switch is off by default.

“Does our data go to a third party?” No. The assistant runs on an on-premise rule-based parser; the LLM fallback is opt-in and off. Pattern Lab trains locally with scikit-learn. Nothing leaves the institution.

“What if a supervisor disputes a suggestion?” Open the score breakdown. Every point is attributed to a named factor - research area 45, topic overlap up to 25, capacity up to 20, track record up to 10.

“How hard is a HESA spec change?” Clone last year's profile and remap the changed fields. The platform refuses sign-off until every mandatory field is mapped. No release required.

“Who can see student data?” Administrators and PGR administrators broadly; supervisors only their own caseload; students only themselves; executives not at all. Enforced in the query, tested, and every write is audited.

Closing the demo

THE THREE CLAIMS TO LAND, AND THE EVIDENCE YOU JUST SHOWED

CLAIM	EVIDENCE
One person, one record, whole life	The applicant became a student against the same person id in one transaction, and Marcus Bell holds student and employee identities with overlapping dates. 63 of 343 researchers are also employees.
Nothing is a black box	Nine named funding rules returning their dates; supervisor matching attributing every point; a model card computing its own limitations; per-student prediction factors in percentage points.
Governance is enforced, not documented	The Executive refused a student record; a trainer cannot approve their own model; Computer Science cannot be deleted while 399 records point at it; a statutory profile cannot be signed off with 15 mandatory fields unmapped.

WHAT IS DELIBERATELY NOT BUILT

> This is **not** a grants management system, **not** finance or payroll, **not** an HR system and **not** an undergraduate SIS. It integrates with those rather than replacing them. Being straight about the boundary earns more credibility than claiming everything - and it is what makes the funding-lineage claim believable, because the platform holds the relationship and finance holds the money.

IF THE ROOM GOES QUIET, ASK THEM THIS

> "Today, if a funder asked which researchers a particular award has paid for over the last three years - how long would that take you, and who would have to do it?" Then show the relationship map again.

STRONGEST ENHANCEMENT CANDIDATES

#	ENHANCEMENT	EFFORT
1	Fix the active-funding predicate so the dashboard tile and the risk rule test status and date coverage rather than `valid_to IS NULL`	Very small
2	SSO against the institution's identity provider - the configuration hooks already exist	Small
3	Ship the current HESA specification as a starter statutory profile	Small
4	Permission-filtered navigation so the menu matches the entitlement	Small
5	Dismiss a funding finding with a recorded justification, and auto-raise a task for error-severity findings	Small
6	Printable student summary for progression panels	Small
7	Two-way finance integration so payment status flows back	Medium
8	Student self-service: milestone evidence upload, meeting confirmation	Medium
9	Scheduled monitoring, alerting and challenger models in Pattern Lab	Medium
10	Role designer so institutions compose their own roles from the 23 permissions	Medium
11	Retention schedule, subject-access export and erasure - needs institutional policy input first	Medium
12	Department scoping across dashboards, analytics and the relationship map	Medium

> **Going live:** two virtual machines, PostgreSQL, an SMTP relay and a certificate. TLS, encryption at rest, rate limiting, backups and CI scanning are all open and all deployment concerns rather than application changes. The infrastructure requirements and deployment guide are separate documents in `docs/`.